

Car Sounds: Real-Time Car Sound Classification with Quantized Neural Networks

On the Arduino Nano 33 BLE Sense Rev2

Affiliations

BDA 602 Machine Learning Engineering, Spring 2026

Professor Ryan Lafler, M.Sc.

San Diego State University

Authors

Robert Ashe – rashe7414@sdsu.edu (Team Leader)

Rohini Panikara – rpanikara5304@sdsu.edu

Shivani Shankar – sshankar6566@sdsu.edu

Yuvaraj Murugan – ymurugan0059@sdsu.edu

GitHub Repository

https://github.com/nthPerson/Car_Sounds_Classification

Abstract

Automotive fault diagnosis traditionally relies on expensive OBD-II scanners or the trained ear of an experienced mechanic, yet many common faults like worn brake pads, low oil pressure, and ignition failures produce characteristic acoustic signatures that a machine learning classifier could detect using only a microphone and a low-cost microcontroller. This paper investigates whether a quantized neural network can perform real-time car sound classification on the Arduino Nano 33 BLE Sense Rev2, an edge device constrained to a 64 MHz Cortex-M4F processor, 1 MB of flash, and 256 KB of SRAM. We train and evaluate five model architectures – Random Forest, SVM, a 2D-CNN, a 1D-CNN, and a depthwise separable CNN (DS-CNN), plus their int8 quantized variants – on the Car Diagnostics Dataset (1,386 WAV files spanning 13 conditions across three operational states: braking, idle, and start-up), organized into a three-tier classification taxonomy culminating in a six-class state-aware target task. Among all candidates, the depthwise separable convolutional neural network (DS-CNN) with post-training quantization (PTQ) achieves the best balance of accuracy and efficiency, reaching a Tier 2 macro F1 of 0.7835 with only 8,166 parameters and a 21.3 KB model footprint. Quantization is nearly lossless: no model-tier-method combination produces a statistically significant accuracy change (McNemar’s test, all $p > 0.05$), and quantization-aware training (QAT) confers no benefit beyond PTQ for models at this scale.

Deployed end-to-end on the Arduino – including PDM audio capture, on-device mel-spectrogram extraction via CMSIS-DSP, and int8 TFLite Micro inference – the system occupies 319 KB of flash (33%) and 175 KB of SRAM (68%), completing a full classification cycle in 2,470 ms (1,490 ms audio capture, 197 ms feature extraction, 785 ms inference). A speaker-to-microphone playback evaluation reveals a 30.3% F1 degradation through a Bluetooth speaker (F1 0.5465, accuracy 68.3%) and an 80.3% degradation through laptop speakers, whose frequency response rolls off below 300 Hz; this speaker ablation study confirms that playback hardware quality, not the classifier, is the dominant source of the domain gap. Additional contributions include a real-world noise corpus of 360 samples collected from a 2008 Volkswagen Jetta using the deployment hardware, a data augmentation study showing that synthetic waveform augmentation with class balancing improves DS-CNN F1 by +0.206, and documentation of a CMSIS-NN SIMD toolchain limitation in the Arduino GCC 7.2.1 environment not previously reported in the literature.

Table of Contents

Abstract.....	2
1. Introduction	4
2. Data Sources.....	7
2.1 Source Dataset.....	7
2.2 Classification Taxonomy	7
2.3 Train, Validation, and Test Splits	7
2.4 Data Preprocessing	8
2.5 Feature Extraction.....	8
2.6 Data Augmentation	10
2.7 Real-World Noise Bank.....	10
2.8 Summary of Data Pipeline	12
3. Methodology.....	12
3.1 Classical ML Baselines.....	12
3.2 Neural Network Architectures.....	13
3.3 Data Augmentation Strategy	15
3.4 Quantization.....	16
3.5 On-Device Deployment.....	17
3.6 Evaluation Framework	18
4. Results	19
4.1 Overall Model Comparison (Tier 2)	19
4.2 Classical ML Baselines Across Tiers	21
4.3 Data Augmentation Ablation	22
4.4 Quantization Impact	23
4.5 On-Device Performance.....	25
4.6 End-to-End Playback Evaluation	27
4.7 Feature Parity Validation.....	30
5. Analysis	31
5.1 Classical ML Baseline Analysis.....	31
5.2 Data Augmentation Analysis	32
5.3 Quantization Analysis.....	33
5.4 Deployment Performance Analysis.....	34
5.5 Playback Test Analysis.....	35
6. Discussion and Industry Applications.....	37

6.1 Key Findings.....	37
6.2 Comparison with Related Work.....	41
6.3 Industry Applications.....	41
6.4 Deployment Recommendations.....	42
6.5 Ethical Considerations.....	43
7. Conclusion and Future Work.....	43
7.1 Summary of Findings.....	43
7.2 Limitations.....	45
7.3 Future Work.....	46
7.4 Concluding Remarks.....	47
8. Author Bios	47
9. References.....	48
10. Appendix.....	49
10.1 GitHub Repository.....	49
10.2 Repository Structure.....	49

1. Introduction

Modern vehicles produce characteristic acoustic signatures under different operating conditions. A healthy engine at idle sounds distinctly different from one suffering low oil pressure; normal brake engagement produces a different acoustic profile than worn brake pads; and a clean ignition start-up is audibly distinguishable from a dead battery or faulty ignition coil. For decades, experienced mechanics have relied on these auditory cues as a primary diagnostic tool, using their trained ears to identify faults that might otherwise require expensive sensor suites or OBD-II scanners to detect (Delvecchio et al., 2018). The question this paper investigates is whether machine learning can replicate and democratize that skill – and, critically, whether it can do so on hardware cheap enough and small enough to be embedded directly in a vehicle.

The automotive industry is undergoing a rapid transformation toward predictive maintenance and connected vehicle architectures. The global predictive maintenance market for vehicles was valued at approximately USD 5.48 billion in 2025, with projections indicating growth to over USD 23.39 billion by 2034 at a compound annual growth rate of 17.5% (Global Market Insights, 2025a). Within the automotive predictive maintenance sensor sub-segment, engine health monitoring alone accounts for roughly 34% of the predictive maintenance sensor market, reflecting the centrality of powertrain diagnostics in vehicle reliability (Global Market Insights, 2025b). This shift is being driven by the proliferation of IoT-enabled sensors, the maturation of AI-powered analytics, and increasing consumer demand for lower maintenance costs and higher vehicle uptime. Sound-based

diagnostics represent a particularly compelling modality within this landscape: microphones are inexpensive, non-invasive, and can capture a wide range of fault signatures – from serpentine belt squeals to ignition misfires – without requiring physical contact with engine components (Nasim et al., 2023; Suman et al., 2022).

Recent work in audio-based vehicle fault detection has demonstrated the viability of machine learning for this task. Akbalık et al. (2024) achieved over 92% accuracy classifying engine faults using MFCC features and an Extreme Learning Machine classifier across multiple vehicle models. Nasim et al. (2023) proposed an intelligent sound-based early fault detection system that first performs binary classification (healthy vs. faulty) before identifying specific fault types. Other studies have explored convolutional neural networks operating on mel-spectrograms for real-time vehicle sound detection (Wang et al., 2022). However, the overwhelming majority of this work assumes server-side inference with GPUs or, at most, edge devices with substantially more computational resources than a microcontroller. The deployment of trained audio classifiers on microcontroller units (MCUs), including the full pipeline of on-device audio capture, feature extraction, and quantized neural network inference, remains underexplored.

This deployment gap matters because many of the practical advantages of audio-based diagnostics are only fully realized at the edge. Cloud-based inference introduces network latency, requires reliable connectivity, and raises privacy concerns since raw audio must be transmitted off-device. By contrast, on-device inference processes audio locally and never transmits it, preserving privacy by design. It operates in environments without network access – a common scenario in repair shops, rural areas, and moving vehicles – and provides near-instantaneous classification results. These advantages align with the broader paradigm of Tiny Machine Learning (TinyML), which brings machine learning capabilities to severely resource-constrained edge devices operating with less than 1 MB of flash memory and limited SRAM (Lin et al., 2023; Abadade et al., 2023). TinyML systems can achieve inference latencies as low as 0.18–300 ms while consuming milliwatts of power, enabling always-on intelligent sensing in battery-powered or energy-harvesting devices (Heydari & Mahmoud, 2025). Key enabling techniques include model quantization – converting 32-bit floating-point weights to 8-bit integers – along with pruning and knowledge distillation, which can achieve compression ratios of up to 49× while preserving acceptable accuracy (Abadade et al., 2023).

Despite these advances, deploying an audio classifier on a typical MCU presents severe engineering constraints. The target hardware for this study is the Arduino Nano 33 BLE Sense Rev2, built around the Nordic nRF52840 system-on-chip featuring an Arm Cortex-M4F processor clocked at 64 MHz. The board provides 1,024 KB of flash memory (983 KB usable) and 256 KB of SRAM – resources that must be shared among the operating system, runtime libraries, audio capture buffers, feature extraction workspace, and the neural network model itself. After accounting for these system-level requirements, the effective budget for the model is approximately 150 KB of flash for weights and graph structure, and 80 KB of SRAM for the TensorFlow Lite Micro tensor arena during inference. The board’s on-board PDM MEMS microphone, with 64 dB signal-to-noise ratio and a practical sample rate of 16 kHz, provides the audio input. These constraints are representative of the broader class of Cortex-

M4 devices that constitute the majority of deployed microcontrollers worldwide, making findings on this platform broadly applicable.

This paper presents an end-to-end investigation of automotive audio classification under these constraints. We train and evaluate multiple model architectures, including classical machine learning baselines (Random Forest, SVM), a 2D convolutional neural network operating on log-mel spectrograms, a 1-D CNN operating on MFCC features, and a depthwise separable CNN (DS-CNN) on the Car Diagnostics Dataset, a publicly available collection of 1,386 labeled WAV recordings spanning 3 operational states (braking, idle, and start-up) and 13 diagnostic conditions (Ragaie, n.d.). We organize these conditions into a hierarchical taxonomy, with our primary deployment target being a 6-class state-aware scheme (Tier 2) that distinguishes normal operation from fault conditions within each operational state. We apply both post-training quantization (PTQ) and quantization-aware training (QAT) to convert float32 models to full-integer int8 representation, deploy the best-performing quantized model on the Arduino Nano 33 BLE Sense Rev2 using TensorFlow Lite for Microcontrollers with CMSIS-NN optimized kernels, and evaluate the complete system across four dimensions: PC-based classification accuracy, quantization impact analysis, on-device performance benchmarking, and end-to-end system evaluation using speaker-based audio playback.

The study is organized around four research questions:

1. **Feasibility.** Can a quantized neural network classify car diagnostic sounds on a Cortex-M4 microcontroller? Does the model fit within the memory budget, and does it run in real time?
2. **Quantization impact.** What is the accuracy cost of int8 quantization for small audio classification models, and is there a statistically significant difference between post-training quantization and quantization-aware training?
3. **Domain gap.** How does the speaker-to-microphone domain gap (the acoustic difference between evaluating a model on PC with original recordings versus playing those recordings through a speaker and capturing them with the on-board microphone) affect real-world deployment accuracy?
4. **Data augmentation.** Does training with real-world noise augmentation improve model robustness for deployment compared to synthetic augmentation alone?

These questions are designed to address not only whether on-device automotive audio classification is technically feasible, but also to provide quantitative guidance on the practical trade-offs involved in deploying such a system. The contribution of this work to BDA 602 is an end-to-end pipeline – from raw audio preprocessing through model training, quantization, and microcontroller deployment – accompanied by a rigorous four-dimensional evaluation framework that captures both algorithmic performance and real-world deployment characteristics.

The remainder of this paper is organized as follows. **Section 2** describes the data sources, including the Car Diagnostics Dataset, the preprocessing pipeline, the data augmentation

strategy, and the noise collection process. **Section 3** presents the methodology, covering model architectures, quantization procedures, the deployment pipeline, and the evaluation framework. **Section 4** provides analysis and interpretation of the experimental observations. **Section 5** reports quantitative results in tabular and graphical form. **Section 6** discusses implications for industry applications and deployment recommendations. **Section 7** concludes with answers to the research questions, limitations, and directions for future work. **Section 8** lists author biographies and **Section 9** lists references.

2. Data Sources

2.1 Source Dataset

This study uses the Car Diagnostics Dataset published on Kaggle by Malak Ragaie (Ragaie, n.d.), which contains 1,386 WAV recordings of automobile sounds spanning three operational states: braking, idle, and startup. The dataset captures 13 distinct conditions, including normal operation and various fault modes. Braking samples include normal braking and worn brakes. Idle samples encompass normal engine idle, low oil level, power steering fault, serpentine belt fault, and four combined-fault conditions (pairwise and triple combinations of the three single faults). Startup samples cover normal startup, bad ignition, and dead battery. The raw recordings exhibit heterogeneous sample rates (24 kHz, 44.1 kHz, and 48 kHz), and their durations differ slightly by operational state: braking and idle clips are approximately 1.5 seconds, while startup clips are approximately 1.7 seconds.

2.2 Classification Taxonomy

To support evaluation at multiple levels of diagnostic granularity, three hierarchical classification tiers were defined over the 13 source conditions.

Tier 1 (Binary) collapses all conditions into two classes – Normal versus Anomaly – providing a coarse screening objective. **Tier 2 (State-Aware, 6 classes)** serves as the primary deployment target and distinguishes normal operation from fault conditions within each operational state: Normal Braking, Braking Fault, Normal Idle, Idle Fault, Normal Start-Up, and Start-Up Fault. Each fault class aggregates all fault conditions for its state; for example, Idle Fault includes low oil, power steering fault, serpentine belt fault, and the combined-fault recordings. **Tier 3 (Individual Faults, 9 classes)** assigns a unique label to each single-fault condition (e.g., Worn Brakes, Low Oil, Bad Ignition), enabling fine-grained diagnosis. Because the combined-fault idle recordings do not map to a single fault, they are excluded from Tier 3 evaluation (assigned a sentinel label of -1) but are retained in Tiers 1 and 2, where they contribute to the Idle Fault aggregate class.

2.3 Train, Validation, and Test Splits

The 1,386 recordings were partitioned into training, validation, and test sets at a 70/15/15 ratio using stratified random sampling over the 13 source conditions, with a fixed random seed of 42 for reproducibility. This stratification ensures that every condition (including minority classes) appears in all three sets. The resulting splits contain 970 training samples,

208 validation samples, and 208 test samples. The validation set was used for hyperparameter tuning and early stopping during neural network training; the test set was reserved for final evaluation and used only once per model.

The Tier 2 (State-Aware, 6-classes) class distribution in the training set reveals substantial imbalance. Idle Fault, which aggregates six underlying conditions, dominates with 552 samples (56.9%), while Normal Start-Up constitutes the smallest class with just 43 samples (4.4%). The resulting imbalance ratio of 12.8:1 presents a significant challenge for classifier training and motivated both class-weighted loss functions and the data augmentation strategy described in Section 2.6. The remaining training-set class counts are Normal Idle (185, 19.1%), Start-Up Fault (83, 8.6%), Normal Braking (54, 5.6%), and Braking Fault (53, 5.5%). Validation and test distributions mirror these proportions due to the stratified split.

2.4 Data Preprocessing

All audio files underwent a four-step standardization pipeline to ensure uniform input dimensions across the dataset and compatibility with the deployment target hardware.

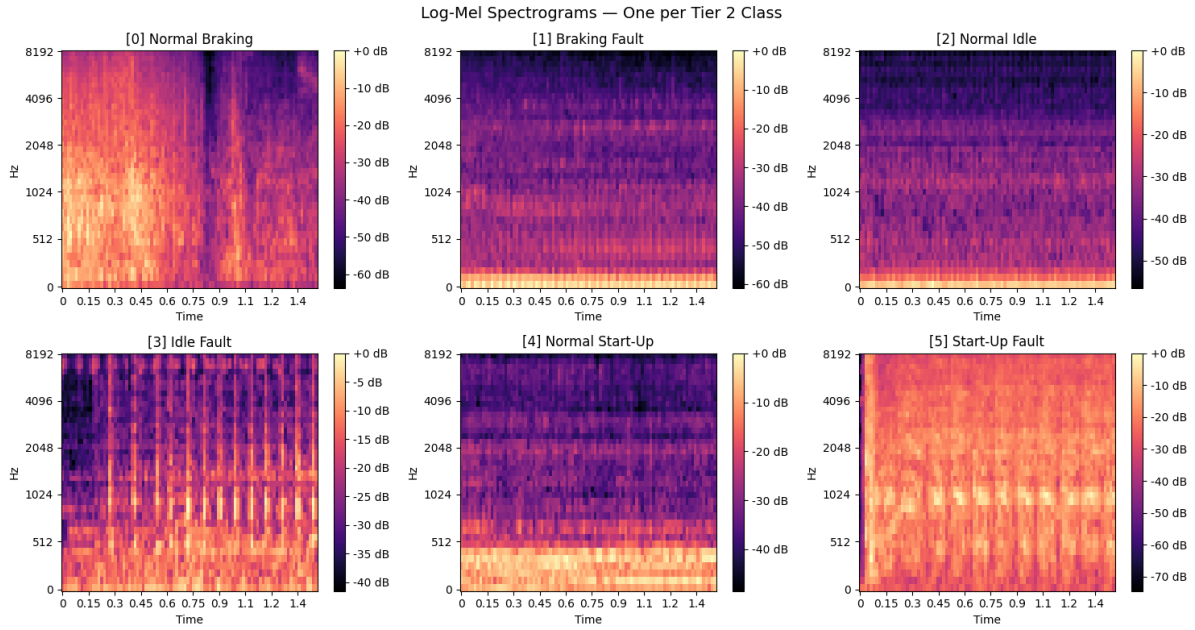
First, every recording was resampled to 16 kHz mono, matching the sample rate of the PDM microphone on the Arduino Nano 33 BLE Sense Rev2 deployment board. Second, each waveform was peak-normalized by dividing by its maximum absolute amplitude, scaling all signals to the range $[-1, 1]$. Third, durations were standardized to exactly 1.5 seconds (24,000 samples): clips shorter than 1.5 seconds were zero-padded, while the 1.7-second startup recordings were center-cropped to 1.5 seconds, preserving the acoustically informative central portion of the ignition event. These preprocessing parameters were serialized to a configuration file (`data/preprocessing_config.json`) to ensure reproducibility across pipeline stages.

2.5 Feature Extraction

Three complementary feature representations were extracted from the preprocessed audio to serve different model architectures.

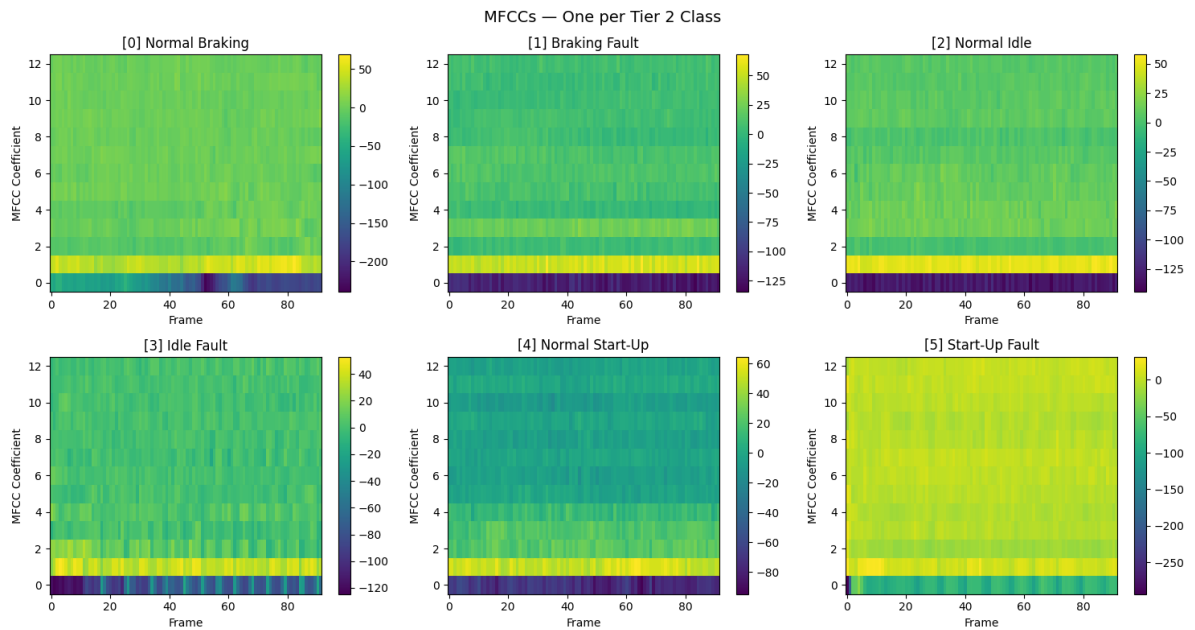
Log-Mel Spectrograms. For the 2D Convolutional Neural Network (M2) and Depthwise Separable CNN (M6), log-mel spectrograms were computed using a 512-point FFT with a hop length of 256 samples and a 40-band mel filterbank spanning 20 Hz to 8,000 Hz. The resulting power spectrograms were converted to decibel scale using a reference-maximum normalization with an 80 dB dynamic range floor. Each spectrogram has shape (40, 92) – 40 mel bands by 92 time frames – and was expanded to (40, 92, 1) for compatibility with 2D convolutional input layers. Per-band z-score normalization was applied using mean and standard deviation statistics computed exclusively from the training set, ensuring no information leakage from the validation or test partitions. **Figure 1** shows log-mel spectrogram examples of samples from each of the 6 classes in Tier 2.

Figure 1: Log-Mel Spectrogram Examples for Each Tier 2 Class



Mel-Frequency Cepstral Coefficients (MFCCs). For the 1-D CNN (M5), 13 MFCCs were extracted using the same FFT and mel filterbank parameters as the spectrogram pipeline, yielding feature matrices of shape (92, 13) – 92 time frames by 13 coefficients. Per-coefficient z-score normalization was again computed from training-set statistics only. **Figure 2** shows example MFCC representations of samples belonging to the 6 Tier 2 classes.

Figure 2: Mel-Frequency Cepstral Coefficient (MFCC) Examples for Each Tier 2 Class



Classical ML Feature Vectors. For the Random Forest and SVM baselines (M1), a 441-dimensional feature vector was computed for each clip. Sixty-three frame-level features

were first extracted: 13 MFCCs, 13 delta MFCCs, 13 delta-delta MFCCs, spectral centroid, spectral bandwidth, spectral rolloff (85th percentile), spectral contrast across 7 sub-bands, RMS energy, zero-crossing rate, and 12 chroma bins. Seven aggregation statistics (mean, standard deviation, minimum, maximum, median, skewness, and kurtosis) were then computed over the time axis for each of the 63 frame-level features, yielding $63 \times 7 = 441$ dimensions per sample.

All normalization statistics were computed separately for each training condition (A through D, described in Section 2.6) to account for distributional shifts introduced by augmentation. Separate statistics files were maintained and applied consistently to validation and test sets at inference time to avoid data leakage.

2.6 Data Augmentation

The severe class imbalance in the training set (12.8:1 ratio) and the modest dataset size (970 training samples) necessitated a data augmentation strategy with two objectives: balancing class frequencies and increasing the effective training-set diversity. An ablation study across four training conditions was designed to isolate the contributions of synthetic augmentation and real-world noise injection.

Condition A (Original) uses the unaugmented 970-sample training set with its natural class imbalance. **Condition B (Synthetic Augmentation)** applies five waveform-domain augmentations – time shift (± 100 ms, 50% probability), additive Gaussian noise (SNR 5–25 dB, 50%), pitch shift (± 2 semitones, 30%), speed perturbation ($0.9\text{--}1.1\times$ rate, 30%), and random gain ($0.7\text{--}1.3\times$ amplitude, 50%) – with per-class multipliers calibrated to bring every Tier 2 class to approximately 550 samples, matching the largest class (Idle Fault at 552). The resulting balanced training set contains 3,317 samples. **Condition C (Noise-Augmented)** applies the same waveform augmentations as Condition B, then additionally injects real-world noise (described in Section 2.7) into 100% of the augmented samples at signal-to-noise ratios uniformly sampled from $\{5, 10, 15, 20\}$ dB, also yielding 3,317 samples. **Condition D (Combined)** merges the augmented samples from Conditions B and C, producing 5,664 training samples. To avoid duplicating the 970 original recordings, Condition D retains the full Condition B set (970 originals plus 2,347 synthetic augmentation copies) and appends only the 2,347 augmented-only copies from Condition C.

All augmentations were applied exclusively to the training set. The validation and test sets remained unaugmented throughout, ensuring fair comparison across conditions. Each augmented copy used a unique random seed for reproducibility.

2.7 Real-World Noise Bank

A distinguishing element of this study’s data pipeline is the collection of a purpose-built noise bank recorded with the same hardware targeted for deployment. The Car Diagnostics Dataset was produced under controlled recording conditions, but the deployment environment – an Arduino microcontroller mounted in or near a vehicle – will encounter engine rumble, road noise, wind, HVAC airflow, and ambient urban sound. Bridging this domain gap required noise samples that capture not only the spectral characteristics of real

automotive environments but also the frequency response, noise floor, and quantization artifacts of the deployment microphone itself.

Collection hardware and protocol. All noise samples were recorded on March 22, 2026, using an Arduino Nano 33 BLE Sense Rev2 with its on-board PDM MEMS microphone – the identical hardware designated for inference. The Arduino was positioned either under the hood (near the engine bay) or inside the cabin of a 2008 Volkswagen Jetta, depending on the noise category. A custom Arduino sketch captured 1.5-second clips at 16 kHz and transmitted each as raw 16-bit integer binary data over a serial connection at 115,200 baud to a companion Python receiver script running on a laptop, which saved each clip as a WAV file and logged metadata (timestamp, label, sample rate, sample count, duration, and RMS energy) to a structured collection log. **Figure 3** shows the installed locations for the Arduino used during the collection of the realistic noise data.

Figure 3: Installed locations of the Arduino Nano 33 BLE Sense Rev2 during realistic noise data collection under hood (left) and inside cabin (right) of a 2008 Volkswagen Jetta.



Noise categories. A total of 360 noise samples were collected across 23 source labels, which were subsequently grouped into six acoustic categories for augmentation purposes. Engine Mechanical (103 samples) captures ambient under-hood sound, cold and warm engine idle, varying RPM, and cooling fan activity. HVAC (30 samples) spans three fan speed settings recorded under the hood. Road/Driving (60 samples) covers low-speed residential, moderate-speed city, and rough-road driving, all recorded under the hood. Cabin/Driving (60 samples) records interior cabin sound during city and highway driving with windows open and closed. Wind (20 samples) includes windy-day ambient and windy driving conditions. Ambient/Stationary (87 samples) encompasses parking lot, quiet residential, stoplight, busy street, and busy highway recordings from inside the cabin.

Quality validation. All 360 samples passed automated validation checks: every file confirmed a sample rate of exactly 16,000 Hz and a duration of exactly 1.5 seconds, with no clipping detected and no silent recordings (all clips exceeded an RMS energy threshold of 0.001).

Operational-state-aware mixing. During noise injection (Condition C), noise samples were not mixed randomly but were selected according to an operational-state relevance mapping. Braking samples received noise from the Engine Mechanical, Road/Driving, Cabin/Driving, Wind, and HVAC groups. Idle samples were mixed with Engine Mechanical, HVAC, Ambient/Stationary, and Wind noise. Startup samples received Ambient/Stationary, Wind, and HVAC noise. This mapping ensures that each training sample encounters noise types realistic for its operational context. The signal-to-noise ratio for each mixture was uniformly sampled from {5, 10, 15, 20} dB, producing a range from substantially corrupted (5 dB) to nearly clean (20 dB) conditions.

2.8 Summary of Data Pipeline

The complete data pipeline transforms 1,386 heterogeneous WAV recordings into standardized, feature-extracted, and optionally augmented training inputs across four experimental conditions. Preprocessing normalizes all audio to 16 kHz mono, 1.5-second clips. Three feature representations (log-mel spectrograms, MFCCs, and 441-dimensional classical feature vectors) serve the five model architectures evaluated in this study. Data augmentation addresses the 12.8:1 class imbalance through targeted oversampling with waveform perturbations, while a purpose-built noise bank of 360 real-world automotive recordings – collected with the deployment hardware itself – enables domain-realistic noise injection. Strict separation of training, validation, and test partitions is maintained at every stage: normalization statistics are computed from training data only, augmentation is applied to training data only, and the test set is used exactly once per model for final evaluation. With the data pipeline and experimental conditions established, we turn next to the machine-learning methodology built on top of these inputs.

3. Methodology

This section describes the experimental pipeline used to train, quantize, deploy, and evaluate audio classification models for real-time automotive fault detection on a microcontroller. The methodology spans five stages: classical machine learning baselines, neural network architecture design and training, data augmentation, integer quantization for edge deployment, and a four-dimensional evaluation framework that captures both PC-based accuracy and real-world on-device performance. The overall pipeline is implemented in Python using scikit-learn for classical baselines and TensorFlow/Keras for neural network training, with TensorFlow Lite Micro and CMSIS-NN handling on-device inference on the Arduino.

3.1 Classical ML Baselines

Two classical machine learning algorithms were trained as non-deployable baselines to establish reference performance using hand-crafted features.

Random Forest. A scikit-learn `RandomForestClassifier` was tuned via `RandomizedSearchCV` with 5-fold stratified cross-validation over 100 iterations. The search space covered `n_estimators` in {100, 200, 500}, `max_depth` in {10, 20, 30, None},

`min_samples_split` in {2, 5, 10}, `min_samples_leaf` in {1, 2, 4}, `max_features` in {"sqrt", "log2", 0.3}, and `class_weight` in {"balanced", "balanced_subsample"}.

Support Vector Machine. A scikit-learn SVC was tuned via GridSearchCV with 5-fold stratified cross-validation. Two kernel configurations were searched: an RBF kernel with `C` in {0.01, 0.1, 1, 10, 100} and `gamma` in {"scale", "auto", 0.001, 0.01, 0.1}, and a linear kernel with the same `C` range.

Both classifiers received 441-dimensional feature vectors as input. These vectors were constructed by computing 63 frame-level audio descriptors – 13 MFCCs, 13 delta MFCCs, 13 delta-delta MFCCs, spectral centroid, spectral bandwidth, 7-band spectral contrast, spectral rolloff, zero-crossing rate, RMS energy, and 12 chroma bins – and then aggregating each descriptor across time using seven statistics (mean, standard deviation, minimum, maximum, median, skewness, and kurtosis). A StandardScaler was fit on the training set only and applied to all splits. Feature importance was assessed using the Random Forest's Gini-based `feature_importances_` attribute, and a reduced-feature experiment retrained the Random Forest on only the top 50 most important features to quantify redundancy in the 441-dimensional space.

In addition to these supervised baselines, the feature-importance procedure itself served as an unsupervised structural analysis of the 441-dimensional feature space: Gini-based importances were used to rank dimensions, and this ranking informed the top-50 reduced-feature retraining experiment. This approach was taken to satisfy the requirement of an unsupervised component while keeping the end-to-end pipeline aligned with the supervised deployment objective.

3.2 Neural Network Architectures

Three convolutional neural network architectures were designed for deployment on the Arduino Nano 33 BLE Sense Rev2, balancing classification capacity against the microcontroller's constraints of 150 KB model flash and 80 KB tensor arena SRAM. Each architecture was trained across all three taxonomy tiers (Tier 1: 2-class binary; Tier 2: 6-class state-aware; Tier 3: 9-class individual faults). Parameter counts reported below correspond to the Tier 2 configuration.

3.2.1 M2 – Compact 2D CNN (14,566 parameters)

The primary architecture is a three-block 2D CNN operating on log-mel spectrograms of shape (40, 92, 1). Each block consists of a Conv2D layer with 3×3 kernels and same padding, followed by batch normalization, ReLU activation, and 2×2 max pooling; the third block omits max pooling and feeds directly into global average pooling. The three convolutional layers use 16, 32, and 32 filters respectively. Global average pooling is followed by a dropout layer (rate = 0.3, determined by hyperparameter search), and a dense softmax output layer. This architecture was chosen as the primary candidate for its proven effectiveness in audio classification tasks while remaining compact enough for microcontroller deployment.

3.2.2 M5 – 1-D CNN on MFCCs (6,246 parameters)

A lighter alternative operating on MFCC frames of shape (92, 13). The architecture mirrors M2's three-block structure but uses 1-D convolutions along the time axis with kernel sizes of 5, 3, and 3 respectively. Channel counts follow the same 16→32→32 progression, with batch normalization, ReLU, and 1-D max pooling (pool size 2) after each convolutional layer. Global average pooling precedes dropout (rate = 0.2) and a dense softmax layer. This architecture offers the smallest tensor arena footprint (estimated 15–25 KB) and serves as a guaranteed-to-fit fallback.

3.2.3 M6 – Depthwise Separable CNN (8,166 parameters)

This architecture follows the DS-CNN-S design from Zhang, Suda, Lai, and Chandra (2017), adapted to the project's input dimensions. It uses a 32-channel configuration (revised from an initial 64-channel design that would have exceeded the 80 KB SRAM budget). The network begins with a Conv2D layer using a 4×10 kernel with stride (2, 2) to reduce the spatial dimensions of the (40, 92, 1) input to (20, 46, 32). This is followed by four depthwise separable (DS) blocks, each consisting of a 3×3 depthwise convolution (same padding), batch normalization, ReLU, a 1×1 pointwise convolution, batch normalization, and ReLU. The DS blocks maintain the (20, 46, 32) spatial dimensions throughout. Global average pooling and a dense softmax layer complete the classifier. Depthwise separable convolutions factor standard convolutions into spatial (depthwise) and channel-mixing (pointwise) operations, reducing both parameters and multiply-accumulate operations (MACs) while maintaining representational capacity. Critically, CMSIS-NN provides optimized kernels for depthwise convolutions on Cortex-M4, making this architecture a strong candidate for hardware-accelerated inference.

3.2.4 Training Configuration

All neural networks shared a common training configuration. The Adam optimizer was used with sparse categorical cross-entropy loss. Class weights were computed using scikit-learn's `compute_class_weight('balanced')` to mitigate the severe class imbalance in the dataset (12.8:1 ratio between the largest and smallest Tier 2 classes). Training ran for up to 100 epochs with early stopping on validation loss (patience = 15, restoring best weights) and a `ReduceLROnPlateau` scheduler (factor = 0.5, patience = 5, minimum learning rate = 1×10^{-6}). The batch size was 32 and all random seeds were fixed at 42 for reproducibility. A held-out validation set of 208 unaugmented samples was used for all intermediate decisions, and the test set of 208 samples was reserved for final reporting only.

3.2.5 Hyperparameter Search

Hyperparameters were tuned on Tier 2 (the primary deployment target) and then applied to Tiers 1 and 3 without tier-specific optimization to reduce computational demands. For each architecture, a structured grid search was conducted sequentially over learning rate ({0.0005, 0.001, 0.002}) and dropout rate ({0.2, 0.3, 0.5}), selecting configurations by best validation accuracy. The optimal settings were as follows:

- M2 at learning rate 0.002 with dropout 0.2 and no augmentation

- M5 at learning rate 0.001 with dropout 0.3 and default augmentation
- M6 at learning rate 0.002 with no dropout sweep (fixed architecture) and no augmentation.

Notably, the DS-CNN of M6 exhibited training instability on the original imbalanced dataset (validation accuracy collapsed to approximately 23% at learning rates ≥ 0.001) but trained stably after data augmentation and class balancing were applied.

3.3 Data Augmentation Strategy

Four training conditions were constructed for an ablation study to isolate the contributions of synthetic augmentation, class balancing, and real-world noise injection. All conditions shared the same unaugmented validation and test sets (208 samples each), the same model architectures, and the same hyperparameters.

Condition A (Original). The unmodified training set of 970 samples with its original class imbalance (43 to 552 samples per Tier 2 class).

Condition B (Synthetic Augmentation + Class Balancing). Five waveform-domain augmentations were applied stochastically to generate additional training samples: time shift (50% probability, ± 100 ms), additive Gaussian noise (50%, SNR 5–25 dB), pitch shift (30%, ± 2 semitones), speed perturbation (30%, $0.9\text{--}1.1\times$ rate), and random gain (50%, $0.7\text{--}1.3\times$ amplitude). Class-aware augmentation multipliers brought each Tier 2 class to approximately 550 samples, matching the size of the largest class (Idle Fault at 552). Per-class multipliers ranged from $0\times$ (Idle Fault, no augmentation needed) to $12\times$ (Normal Start-Up, the most underrepresented class with only 43 original samples). The resulting training set contained 3,317 samples with an imbalance ratio reduced from 12.8:1 to 1.1:1.

Condition C (Noise-Augmented). The same waveform augmentation pipeline as Condition B, with an additional stage of real-world noise injection applied to 100% of augmented samples. Noise was drawn from a bank of 360 recordings collected with the Arduino Nano 33 BLE Sense Rev2's on-board PDM microphone in and around a 2008 Volkswagen Jetta. The noise bank was organized into six groups (engine mechanical, HVAC, road/driving, cabin/driving, wind, and ambient/stationary) totaling 23 subcategories. Noise injection was operationally state-aware: braking samples received road, driving, and engine noise; idle samples received engine, HVAC, and ambient noise; and startup samples received ambient, wind, and HVAC noise. The signal-to-noise ratio (SNR) for each sample was uniformly drawn from {5, 10, 15, 20} dB, and mixing followed the standard additive formula: $\text{mixed} = \text{signal} + \text{scale} \times \text{noise}$, where the scale factor was computed as $\sqrt{(\text{signal_power} / (\text{noise_power} \times 10^{(\text{SNR}/10))})}$. The resulting training set was also 3,317 samples with the same class distribution as Condition B, enabling a controlled comparison of the isolated effect of noise injection.

Condition D (Combined). The union of Condition B's full dataset (970 originals plus 2,347 synthetic copies) and Condition C's 2,347 noise-augmented copies (excluding the original 970 to avoid duplication), yielding 5,664 training samples. This condition inverted the

original class imbalance – Idle Fault (552 samples, no augmentation) became the smallest class – but `compute_class_weight('balanced')` compensated during training.

Each architecture was trained under its best-performing condition for subsequent quantization: M2 under Condition D, and M5 and M6 under Condition B. All random operations used deterministic seeds derived from a base random state of 42 to ensure full reproducibility.

3.4 Quantization

All neural network models were quantized to int8 representations using two methods to investigate the accuracy-efficiency trade-off central to the project's research questions.

3.4.1 Post-Training Quantization (PTQ)

PTQ was applied using the TensorFlow Lite converter with full-integer int8 quantization. A representative dataset of 200 stratified samples from the training set was used to calibrate activation quantization ranges via min-max estimation. The converter was configured with `supported_ops` restricted to `TFLITE_BUILTINS_INT8` (no float32 fallback), and both input and output types were set to int8. This produced models where all weights, activations, and I/O tensors operated in 8-bit integer arithmetic.

3.4.2 Quantization-Aware Training (QAT)

QAT was performed using the `tensorflow-model-optimization` (tfmot) library v0.8.0. The `quantize_model()` function was applied to the float32 Keras model to insert fake-quantization nodes that simulate int8 rounding during forward passes, allowing the model to adapt its weights to quantization effects during fine-tuning. QAT fine-tuning used a learning rate of $0.1 \times$ the original training rate (M2: 0.0002, M6: 0.0002), with a maximum of 30 epochs, early stopping (patience = 10), and `ReduceLROnPlateau` scheduling. After fine-tuning, the QAT model was converted to int8 TFLite using the same converter settings as PTQ. A notable limitation was that the tfmot library (v0.8.0) does not support Conv1D layers, so M5 could only be quantized via PTQ.

3.4.3 Technical Environment

Quantization required TensorFlow 2.18.0 and the legacy Keras backend (`tf_keras==2.18.0` with the `TF_USE_LEGACY_KERAS=1` environment variable) due to compatibility constraints between tfmot and the TensorFlow 2.18 API. Float32 models were retrained under this legacy backend to produce .h5 files compatible with the quantization pipeline. TensorFlow 2.21 was not viable due to a cuDNN autotuner issue on Ada Lovelace GPUs (hardware used during training).

In total, 15 int8 TFLite models were produced: 9 PTQ models (3 architectures $\times$ 3 tiers) and 6 QAT models (M2 and M6 only, across 3 tiers).

3.5 On-Device Deployment

3.5.1 Hardware Platform

The deployment target was the Arduino Nano 33 BLE Sense Rev2, featuring a Nordic nRF52840 system-on-chip with an ARM Cortex-M4F processor running at 64 MHz, 1,024 KB of flash memory (983 KB usable), 256 KB of SRAM, and an on-board ST MP34DT06JTR PDM MEMS microphone sampling at 16 kHz.

3.5.2 Model Export

The selected deployment model (M6 DS-CNN PTQ, Tier 2) was exported to C header files using a custom script (`src/export_for_arduino.py`). This script generated four headers: model weights as a byte array (21.3 KB), a sparse mel filterbank (490 non-zero weights out of 10,280 possible entries, achieving 95.2% sparsity and reducing flash usage from 40.2 KB to 2.1 KB), per-band normalization statistics (training set mean and standard deviation for z-score normalization), and a 512-point Hann window for FFT windowing.

3.5.3 On-Device Feature Extraction

The on-device feature extraction pipeline was implemented in C++ (`arduino/car_sound_classifier/feature_extraction.cpp`) and executed a five-phase algorithm to transform raw PDM audio into a quantized int8 mel-spectrogram input tensor:

1. **Peak normalization:** The maximum absolute sample value across all 24,000 int16 samples (1.5 seconds at 16 kHz) was identified to establish a reference amplitude.
2. **Frame-by-frame FFT:** The audio was divided into 92 overlapping frames (512-sample window, 256-sample hop). Each frame was windowed with a Hann function and transformed via `arm_rfft_fast_f32()` from the CMSIS-DSP library, followed by power spectrum computation and application of the sparse mel filterbank (40 bands).
3. **Global-max log conversion:** Mel energies were converted to log scale using $10 \times \log_{10}(\text{energy} / \text{max_energy})$, with values clamped to -80 dB. The global-maximum reference made the pipeline scale-invariant, eliminating concerns about FFT magnitude scaling differences between the CMSIS-DSP and librosa implementations.
4. **Z-score normalization:** Per-band normalization using the training set's mean and standard deviation.
5. **Int8 quantization:** Normalized float values were mapped to int8 using the TFLite model's input quantization parameters (scale and zero point), with clamping to the $[-128, 127]$ range.

3.5.4 TFLite Micro Runtime

The TFLite Micro runtime was built from source from the official `tflite-micro` repository with CMSIS-NN optimized kernels, using the `create_tflm_tree.py` tool with `OPTIMIZED_KERNEL_DIR=cmsis_nn` and `TARGET_ARCH=cortex-m4+fp`. The generated source tree was packaged as an Arduino-compatible library (`TensorFlowLite_CMSIS_NN`) with include path fixes for flatbuffers, gemmlowp, ruy, and kissfft dependencies. Additional modifications included injecting `#define CMSIS_NN` into 14 kernel headers and adding ACLE

intrinsic compatibility shims for the Arduino toolchain's GCC 7.2.1 compiler. A `MicroMutableOpResolver<5>` was used to reduce SRAM overhead, registering only the five operations required by the model: Conv2D, DepthwiseConv2D, FullyConnected, Mean (for global average pooling), and Softmax. The tensor arena was efficiently allocated at 65,536 bytes (64 KB), of which 63,556 bytes were used.

3.5.5 Feature Parity Validation

To ensure the on-device feature extraction pipeline produced outputs equivalent to the on-PC Python training pipeline, a feature parity validation was conducted (`src/validate_features.py`). A simulated on-device algorithm was implemented in Python (without `librosa`) and compared against the `librosa`-based training pipeline across 24 test clips (4 per Tier 2 class). Acceptance criteria required cosine similarity ≥ 0.99 and int8 mean absolute error ≤ 2.0 . All 24 clips passed, achieving cosine similarity of 1.0000 and an int8 exact match rate of 96.3%, with the 3.7% of differing values attributable to rounding at quantization boundaries.

3.6 Evaluation Framework

The evaluation framework was organized into four dimensions, designed to capture both the traditional machine learning performance and the deployment-specific characteristics of the system.

3.6.1 Dimension 1 – Classification Performance (PC Evaluation)

All models were evaluated on the held-out test set (208 samples for Tiers 1 and 2; 143 samples for Tier 3) using standard classification metrics: overall accuracy, macro-averaged F1 score, weighted-averaged F1 score, and per-class precision, recall, and F1 scores. Confusion matrices (both raw counts and row-normalized) were generated for all model-tier combinations. For Tier 1 (binary classification), ROC curves and area under the curve (AUC) were additionally computed. The macro F1 score served as the primary ranking metric because it accounts for class imbalance by weighting all classes equally, unlike overall accuracy which can be inflated by strong majority-class performance.

3.6.2 Dimension 2 – Quantization Impact Analysis

This dimension constitutes the central academic contribution of the project. For each quantized model, four metrics were computed to characterize the effect of int8 quantization relative to the float32 baseline: the change in accuracy ($\Delta\text{Accuracy}$), the change in macro F1 score (ΔF1), the prediction agreement rate (the percentage of test samples receiving the same predicted label from both the float32 and int8 models), and the statistical significance of any accuracy difference as assessed by McNemar's test. A McNemar p-value exceeding 0.05 indicates that the float32 and int8 models do not differ significantly in their error patterns. Additionally, confidence distribution shifts were analyzed by comparing the mean and standard deviation of the maximum softmax probability between float32 and int8 predictions.

3.6.3 Dimension 3 – On-Device Performance Benchmarking

Hardware performance was measured directly on the Arduino Nano 33 BLE Sense Rev2. Flash and SRAM utilization were read from the Arduino IDE compilation output. The tensor arena size was determined empirically via `interpreter->arena_used_bytes()`. Latency was measured using `micros()` timers around each pipeline phase: audio capture (fixed at 1,490 ms for 1.5 seconds of PDM recording), feature extraction (mel-spectrogram computation), and model inference (`interpreter->Invoke()`). Multiple inference cycles were averaged to compute mean and standard deviation of latency.

3.6.4 Dimension 4 – End-to-End System Evaluation (Playback Test)

The most realistic evaluation available without an actual malfunctioning vehicle involved playing back the 208 test-set audio clips through external speakers and allowing the Arduino’s on-board PDM microphone to capture and classify them. An automated evaluation script (`src/playback_test.py`) coordinated the process: it sent a READY command over serial, waited for the Arduino to report “Listening...”, played the audio clip through the speaker, then captured the Arduino’s RESULT output over serial. The same classification metrics as Dimension 1 were computed against the ground-truth labels, and the difference between PC and playback accuracy quantified the domain gap introduced by the speaker-microphone signal chain and ambient noise.

A speaker hardware ablation study was conducted using two playback configurations—built-in laptop speakers and an external Bluetooth speaker—to characterize the effect of audio reproduction quality on end-to-end system accuracy. Both configurations used the same 208 test clips, the same Arduino placement, and the same automated evaluation protocol.

4. Results

This section presents the quantitative results of all experiments conducted across the four evaluation dimensions defined in Section 3: classification performance on PC (D1), quantization impact (D2), on-device performance benchmarking (D3), and end-to-end system evaluation via playback testing (D4). **Unless otherwise noted, all results are reported on the held-out test set (n = 208) and use macro-averaged F1-score as the primary comparison metric**, following the rationale established in the evaluation plan: macro F1 weights all classes equally regardless of support, penalizing models that achieve high accuracy by over-predicting the majority class.

4.1 Overall Model Comparison (Tier 2)

The six-class, state-aware taxonomy from Tier 2 serves as the primary deployment target and focus of the evaluation throughout this study. **Table 1** ranks all evaluated models by F1 macro on the Tier 2 test set.

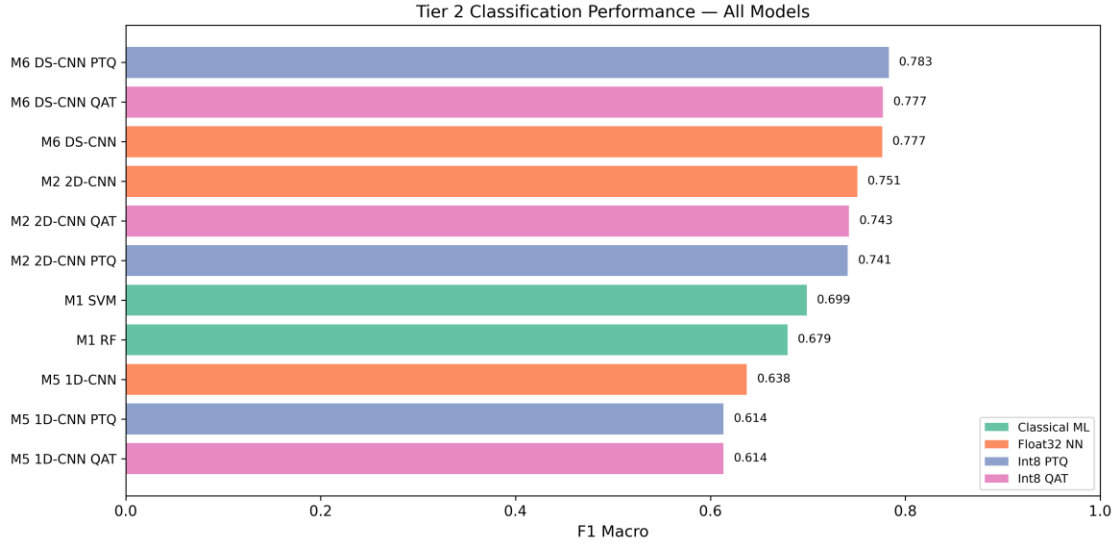
Table 1: Master Comparison of All Models Evaluated at Tier 2 (six-class state-aware classification), Ranked by Macro-Averaged F1-Score

Rank	Model	Type	Training Condition	Accuracy	F1 Macro	F1 Weighted	Size (KB)
1	M6 DS-CNN PTQ	Int8	B	0.8702	0.7835	0.8709	21.3
2	M6 DS-CNN QAT	Int8	B	0.8750	0.7774	0.8720	22.3
3	M6 DS-CNN	Float32	B	0.8702	0.7767	0.8660	246
4	M2 2D-CNN	Float32	D	0.8750	0.7512	0.8729	237
5	M2 2D-CNN QAT	Int8	D	0.8702	0.7426	0.8688	20.5
6	M2 2D-CNN PTQ	Int8	D	0.8654	0.7413	0.8624	20.1
7	SVM	Classical	A	0.8510	0.6992	0.8422	–
8	Random Forest	Classical	A	0.8413	0.6794	0.8272	–
9	M5 1D-CNN	Float32	B	0.8029	0.6377	0.8100	140
10	M5 1D-CNN PTQ	Int8	B	0.8125	0.6138	0.8115	15.2

The depthwise separable CNN (M6 DS-CNN) achieves the highest F1 macro across all model types and quantization methods. Its post-training quantized variant (PTQ, Int8) reaches an F1 macro of 0.7835, making it the top-ranked model overall and the selected candidate for on-device deployment. The M2 2D-CNN achieves a comparable raw accuracy (0.8750 vs. 0.8702), but its lower F1 macro (0.7512) reveals a tendency to over-predict Idle Fault, the majority class (n = 118 of 208 test samples). This disparity between accuracy and F1 macro underscores why macro-averaged F1 is the more appropriate primary metric for this imbalanced dataset.

The **classical ML baselines** (SVM and Random Forest) are competitive on overall accuracy (0.8510 and 0.8413, respectively) but lag substantially behind the neural networks on F1 macro, indicating poorer performance on minority classes such as Normal Start-Up (n = 9) and Braking Fault (n = 11). The M5 1D-CNN, despite being the most compact architecture (15.2 KB quantized), ranks lowest among all models, suggesting that the MFCC feature representation discards spectral information important for distinguishing fault conditions at this granularity. See **Figure 4** for the classification accuracy ranking of all evaluated models and note that M6 DS-CNN PTQ leads the field while requiring only 21.3 KB of flash.

Figure 4: Master Tier 2 Accuracy Comparison Across All Evaluated Models, Ranked by Macro-Averaged F1-Score



4.2 Classical ML Baselines Across Tiers

To contextualize the neural network results, **Table 2** reports the classical ML baseline performance across all three taxonomy tiers (see Section 2.2 for class taxonomy definitions).

Table 2: Classical ML Performance by Taxonomy Tier

Model	Tier	Accuracy	F1 Macro	F1 Weighted	CV F1 Macro
SVM	1	0.9327	0.9163	0.9316	0.9244
Random Forest	1	0.8990	0.8724	0.8965	0.8899
SVM	2	0.8510	0.6992	0.8422	0.7753
Random Forest	2	0.8413	0.6794	0.8272	0.7246
SVM	3	0.7552	0.7112	0.7463	0.7091
Random Forest	3	0.7622	0.7021	0.7521	0.6880
Random Forest (top-50)	2	0.8125	0.6724	0.8124	–

Note: CV F1 Macro reports five-fold stratified cross-validation on the combined training and validation sets.

Both models perform well on the binary Tier 1 task (SVM F1 macro = 0.9163), confirming that the dataset contains a strong normal-vs-anomaly signal in the hand-crafted feature space. Performance degrades substantially at Tier 2, where the gap between accuracy and F1 macro widens – a signature of class-imbalanced misclassification. Interestingly, at Tier 3 (nine classes), the SVM achieves a slightly higher F1 macro (0.7112) than at Tier 2 (0.6992), likely because the Tier 3 formulation excludes combined-fault samples that conflate multiple acoustic signatures into one class. A reduced Random Forest trained on the top-50 features by importance did not improve over the full 441-feature model at the targeted taxonomy

Tier 2 (F1 macro 0.6724 vs. 0.6794), suggesting that the feature space is not dominated by irrelevant dimensions.

4.3 Data Augmentation Ablation

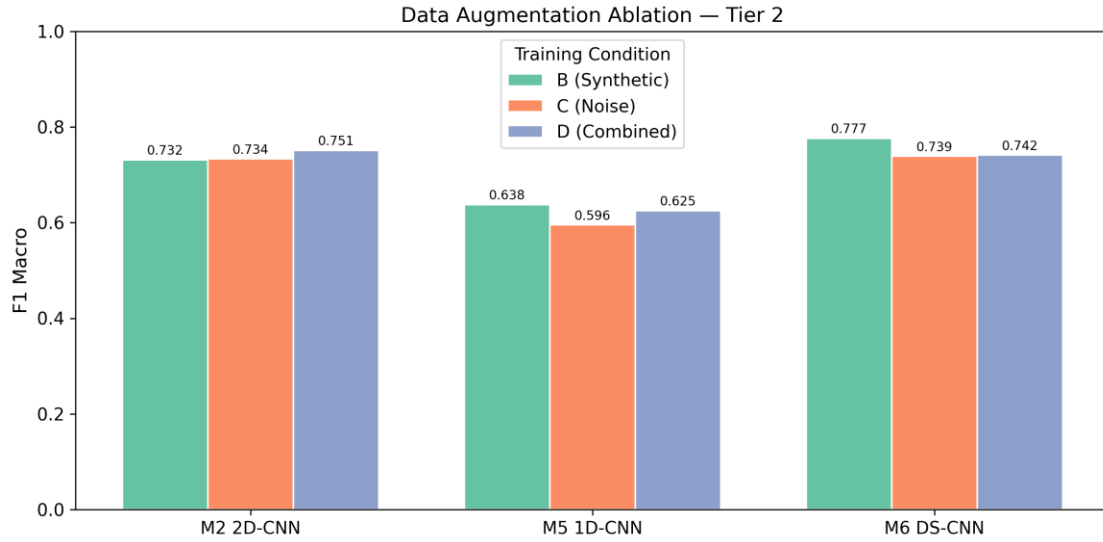
Three training conditions were compared for each neural network architecture at Tier 2 to assess the impact of the data augmentation strategy. Condition B applies waveform augmentations (time shift, Gaussian noise, pitch shift, speed perturbation, random gain) with class-balancing to bring minority classes to approximately 550 samples each (3,317 total). Condition C adds real-world noise injection at SNR levels of 5–20 dB using 360 noise samples recorded on the Arduino hardware. Condition D combines the augmented samples from both B and C (5,664 total). **Table 3** presents the F1 macro scores for each Tier 2 neural network architecture under the three training conditions representing different levels of data augmentation (see Sections 2.6 and 3.3 for augmentation details).

Table 3: F1 Macro by Training Condition and Architecture on Tier 2 Test Set

Architecture	Cond B (Synthetic)	Cond C (Noise)	Cond D (Combined)	Best
M6 DS-CNN	0.7767	0.7388	0.7415	B
M2 2D-CNN	0.7315	0.7335	0.7512	D
M5 1D-CNN	0.6377	0.5956	0.6252	B

The results reveal architecture-dependent interactions with the augmentation strategy. M6 DS-CNN and M5 1D-CNN perform best under Condition B (synthetic augmentation only), while M2 2D-CNN benefits from the larger combined dataset of Condition D. The noise-injected Condition C does not consistently improve clean-test-set performance for any architecture, though it may confer robustness benefits in noisy deployment environments that the clean test set does not capture. M5 shows the sharpest degradation under noise augmentation (F1 macro drops from 0.6377 to 0.5956 under Condition C), consistent with the hypothesis that MFCCs – which operate on a compressed cepstral representation – may already discard frequency bands where real-world noise provides useful regularization signal. **Figure 5** depicts the Tier 2 augmentation ablation showing F1 macro across Conditions B (synthetic), C (noise-augmented), and D (combined) for the three neural network architectures.

Figure 5: Augmentation Ablation at Tier 2



Based on these results, each architecture was carried forward under its best condition for quantization: M6 and M5 under Condition B, and M2 under Condition D.

4.4 Quantization Impact

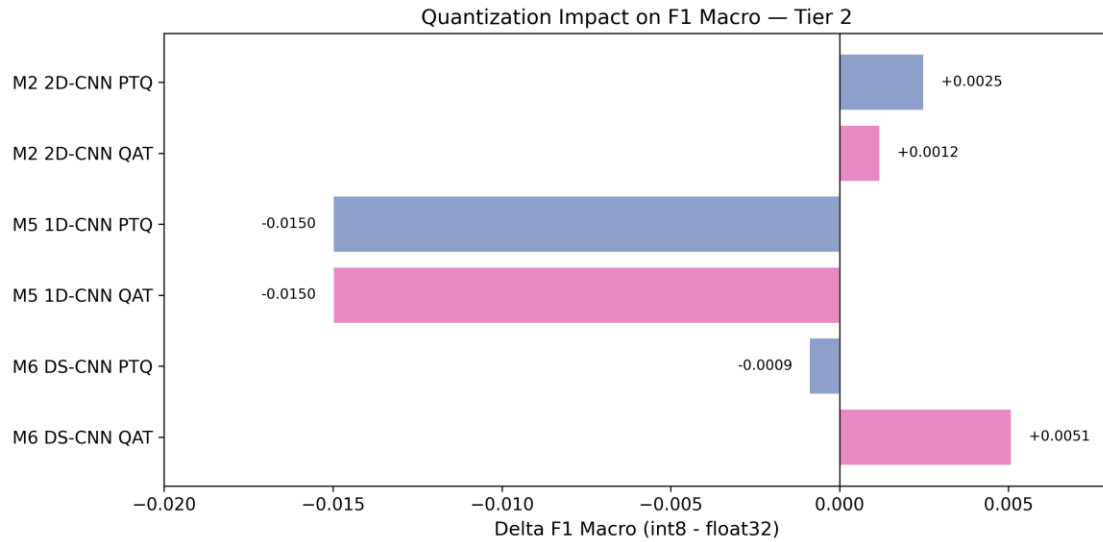
All three neural network architectures were quantized to int8 using both post-training quantization (PTQ) and quantization-aware training (QAT). **Table 4** summarizes the quantization results at Tier 2, where agreement denotes the percentage of test samples receiving identical predictions from the float32 and int8 models. McNemar’s test assesses whether the disagreements are statistically significant, and no combination produced a statistically significant change (all McNemar $p > 0.05$). **Figure 6** shows the impact on F1 scores for each model and quantization method.

Table 4: Quantization Impact on Tier 2 by F1 Macro

Model	Method	Float32		$\Delta F1$	Agreement	McNemar p	Size (KB)
		F1	Int8 F1				
M6 DS-CNN	PTQ	0.7826	0.7835	-0.0009	98.6%	> 0.05	21.3
M6 DS-CNN	QAT	0.7826	0.7774	+0.0051	95.2%	> 0.05	22.3
M2 2D-CNN	PTQ	0.7438	0.7413	+0.0025	99.0%	> 0.05	20.1
M2 2D-CNN	QAT	0.7438	0.7426	+0.0012	95.7%	> 0.05	20.5
M5 1D-CNN	PTQ	0.5987	0.6138	-0.0150	99.0%	> 0.05	15.2
M5 1D-CNN	QAT*	0.5987	0.6138	-0.0150	99.0%	> 0.05	15.2

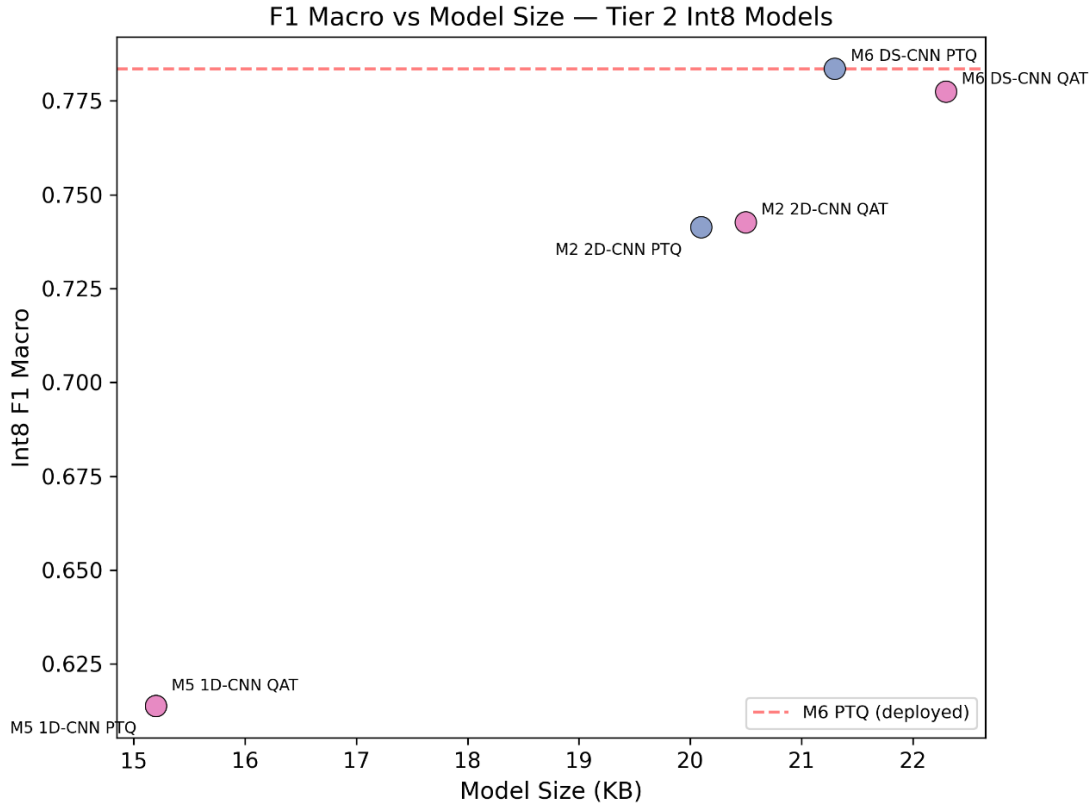
*M5 QAT is identical to M5 PTQ because the TensorFlow Model Optimization Toolkit (tfmot) does not support quantization-aware training for Conv1D layers, so QAT could not be applied to M5.

Figure 6: Quantization Impact ($\Delta F1$) by Model and Method, Compared Against Their Float32 Parents



The central finding is that int8 quantization is effectively lossless for these small audio classification models. No model-method combination exhibits a statistically significant accuracy change (all McNemar $p > 0.05$), and prediction agreement rates range from 95.2% to 99.0%. No model shows substantial F1 degradation, and the largest observed decrease is 0.0051 (M6 QAT). M5 PTQ actually shows a marginal F1 improvement of 0.015 (int8 exceeds float32) – a phenomenon attributable to the regularization effect of reduced numerical precision in small networks. Int8 model sizes range from 15.2 KB to 22.3 KB, representing a 6–16 $\times$ compression from their float32 counterparts, and all fit comfortably within the 150 KB flash budget of the Arduino Nano 33 BLE Sense Rev2. **Figure 7** plots F1 macro against model size for all quantized models, showing that size is not a deterministic factor in predicting performance, and that these small neural networks occupy a dense efficient frontier well within the 150 KB flash budget.

Figure 7: Accuracy Versus Model Size for All Evaluated Int8 Variants



These results indicate that PTQ is sufficient for deployment – QAT provides no measurable benefit over PTQ for any architecture, likely because the small model sizes (6,246–14,566 parameters) and batch normalization folding leave minimal quantization error even without retraining.

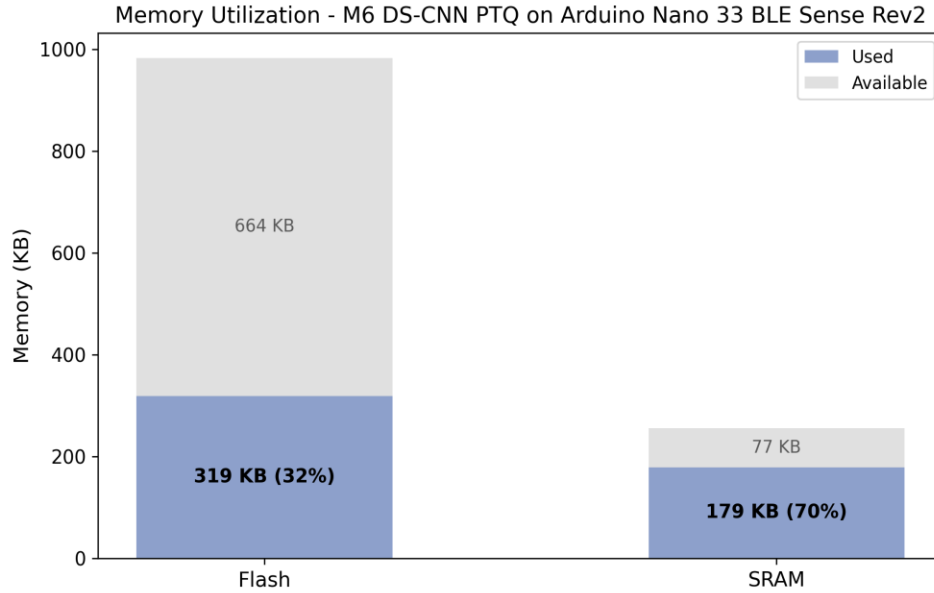
4.5 On-Device Performance

The selected deployment model (M6 DS-CNN PTQ, Tier 2) was deployed on the Arduino Nano 33 BLE Sense Rev2 running TensorFlow Lite for Microcontrollers with CMSIS-NN optimized kernels. **Table 5** reports comprehensive resource utilization and **Figure 8** shows flash and SRAM utilization of the M6 DS-CNN PTQ on the deployment hardware.

Table 5: Resource Utilization of M6 DS-CNN PTQ on the Arduino Nano 33 BLE Sense Rev2.

Resource	Used	Budget	Utilization
Flash	319 KB	983 KB	32.4%
SRAM (static)	179 KB	256 KB	69.9%
Tensor arena	63,556 B	65,536 B allocated (80 KB budget)	97.0% of allocated; 77.5% of budget
SRAM (remaining)	81 KB	–	Available for stack/heap

Figure 8: Flash and SRAM Utilization for the Deployed M6 DS-CNN PTQ Model Versus Hardware Budgets



The model consumes only 32.4% of available flash, leaving substantial room for application logic, BLE stack, and potential multi-model deployment. SRAM utilization is tighter: the tensor arena occupies 63.6 KB of the 80 KB budget (77.5%), and total static SRAM usage reaches 69.9% of the 256 KB available. The remaining 77 KB of SRAM provides adequate headroom for stack and heap operations during inference. **Table 6** reports inference latency broken down by phase for the M6 DS-CNN PTQ model on the deployment hardware.

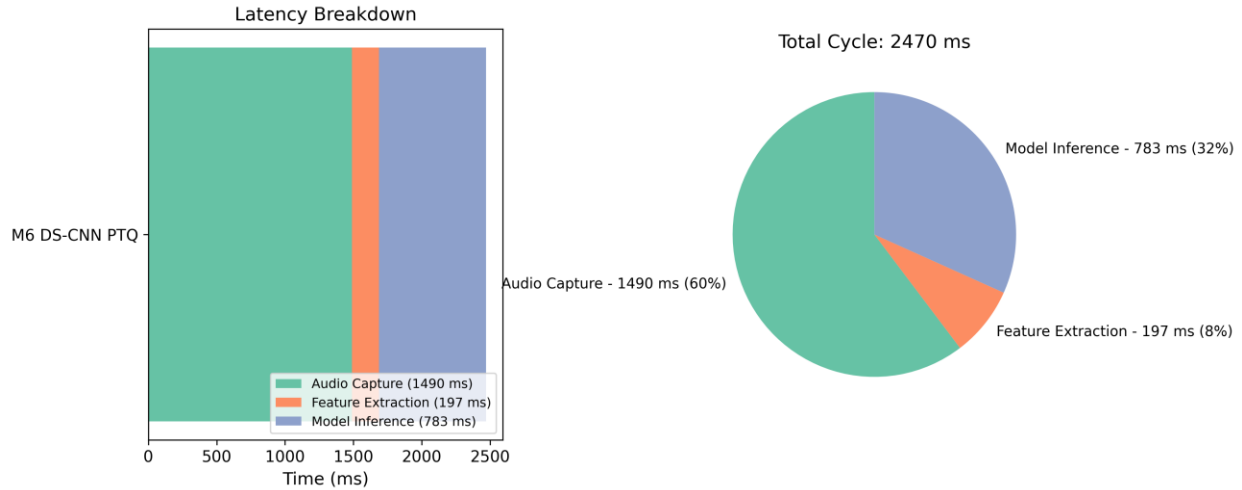
Table 6: Inference Latency Breakdown for M6 DS-CNN PTQ on the Arduino Nano 33 BLE Sense Rev2

Phase	Mean (ms)	Std (ms)	Share
Audio capture	1,490	–	60.3%
Feature extraction	197.4	0.2	8.0%
Model inference	784.9	0.1	31.7%
Total cycle	2,472.5	0.7	100%

A complete classification cycle takes approximately 2.5 seconds, dominated by the 1.49-second audio capture window (1.5 seconds of audio at 16 kHz). Model inference requires 784.9 ms, yielding 8.4 cycles per multiply-accumulate (MAC) operation over the model’s 6,005,952 MACs per inference. This is below the theoretical reference of approximately 10 cycles/MAC for generic int8 operations on Cortex-M4 but above the approximately 3 cycles/MAC achievable with fully optimized CMSIS-NN SIMD kernels. The gap is attributable to the older GCC 7.2.1 toolchain bundled with the Arduino Mbed OS board package, which does not fully exploit the Cortex-M4’s SIMD capabilities. Upgrading to GCC 12+ via PlatformIO or the nRF Connect SDK could reduce inference latency to approximately 282 ms,

bringing the total cycle time below 2 seconds. Nevertheless, for the target use case – a handheld diagnostic tool held near the engine for several seconds – a 2.5-second cycle is operationally acceptable. **Figure 9** provides a latency breakdown for a complete classification cycle, showing the relative contributions of audio capture, feature extraction, and model inference.

Figure 9: Classification Cycle Latency Breakdown



4.6 End-to-End Playback Evaluation

The end-to-end system evaluation (D4) assessed real-world inference by playing the test set audio through external speakers, receiving the sounds with the Arduino’s on-board PDM microphone, and classifying with the deployed M6 DS-CNN PTQ neural network. Three playback configurations were tested: direct PC audio output (baseline), a Bluetooth speaker, and laptop speakers. **Table 7** reports overall classification metrics from the end-to-end playback evaluation.

Table 7: Overall Classification Metrics by Playback Configuration

Metric	PC Int8 (Baseline)	Bluetooth Speaker	Laptop Speakers
Accuracy	0.7837	0.6827	0.2500
F1 Macro	0.7835	0.5465	0.1540
F1 Weighted	0.7835	0.7162	0.2825
Degradation (F1 Macro)	–	–30.3%	–80.3%

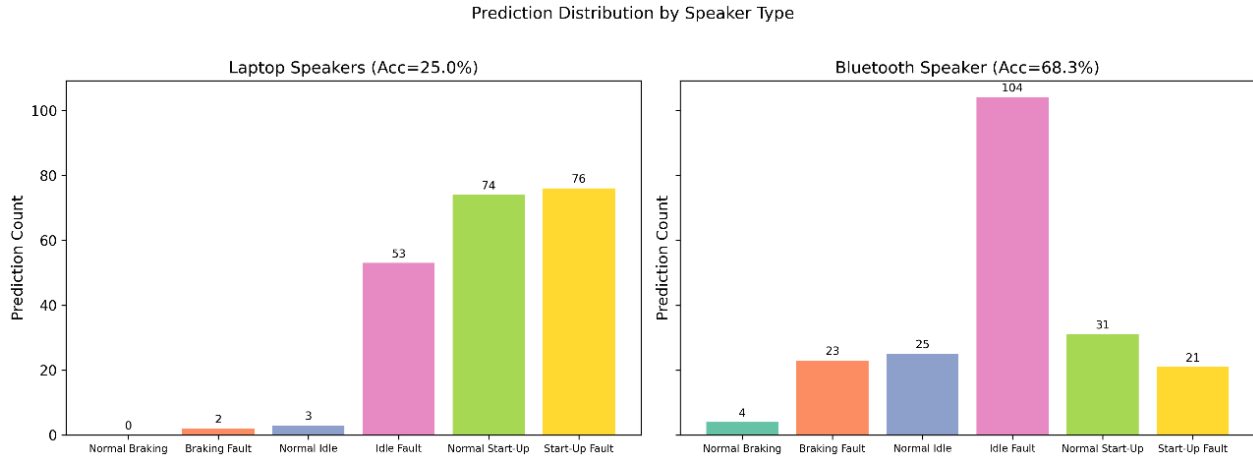
The Bluetooth speaker configuration – the most realistic proxy for field deployment – achieves 68.3% accuracy with an F1 macro of 0.5465, representing a 30.3% degradation from the PC baseline. Laptop speakers produced near-random performance (F1 macro = 0.1540), rendering them unsuitable for this application. The Bluetooth speaker’s relatively high F1 weighted (0.7162) compared to its F1 macro (0.5465) indicates that the degradation is concentrated in minority classes, a pattern confirmed by the per-class recall analysis in

Table 8. The two bar charts in **Figure 10** show the prediction distributions for the Bluetooth and laptop speakers during end-to-end playback evaluation, demonstrating that laptop playback concentrates predictions in startup classes due to low-frequency roll-off.

Table 8: Per-Class Recall for Bluetooth Speaker Playback Versus PC Baseline

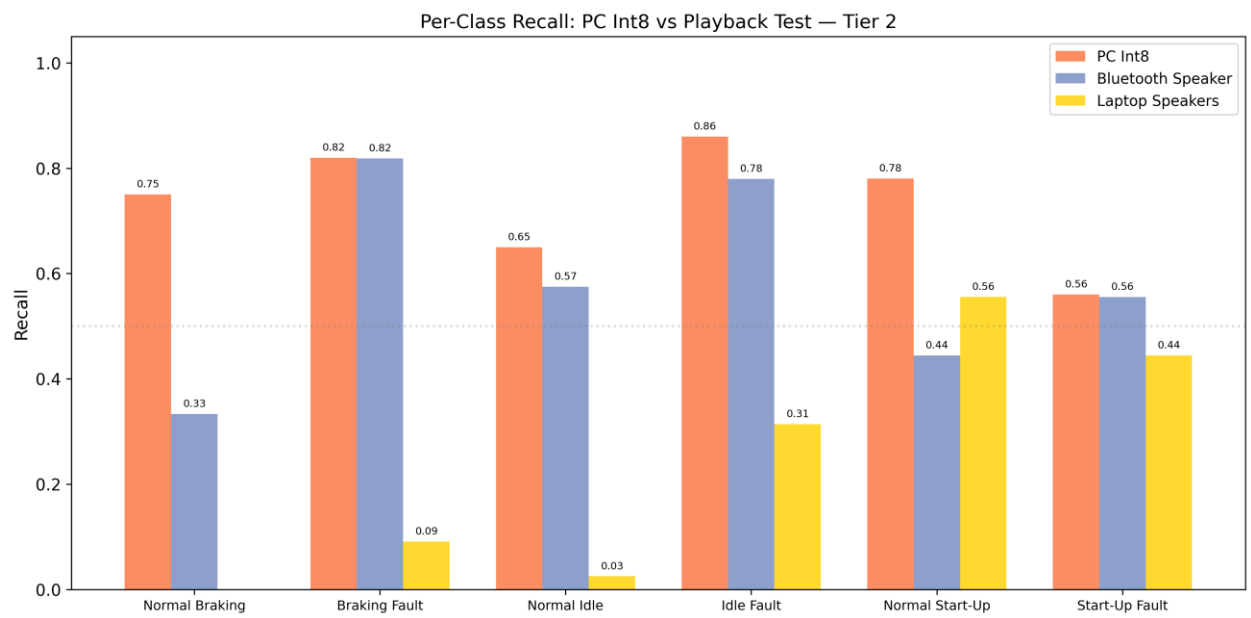
Class	n (test)	PC Int8 Recall	Bluetooth Recall	Δ Recall
Idle Fault	118	0.864	0.780	-0.084
Normal Idle	40	0.675	0.575	-0.100
Start-Up Fault	18	0.556	0.556	0.000
Normal Braking	12	0.750	0.333	-0.417
Braking Fault	11	0.818	0.818	0.000
Normal Start-Up	9	0.778	0.444	-0.334

Figure 10: Speaker Ablation Prediction Distributions Under Bluetooth-Speaker and Laptop-Speaker Playback



The playback degradation is highly class-dependent. Braking Fault and Start-Up Fault retain identical recall through the Bluetooth speaker (0.818 and 0.556, respectively), suggesting that their acoustic signatures are robust to the speaker-microphone domain shift. In contrast, Normal Braking and Normal Start-Up suffer the steepest recall drops (-0.417 and -0.334), likely because normal operating sounds have lower signal energy and subtler spectral features that are more susceptible to distortion from Bluetooth audio compression and speaker frequency response limitations. **Figure 11** shows a per-class recall comparison between the PC direct audio signal baseline and the Bluetooth and laptop speaker playback scenarios, where normal classes degrade most under playback and fault classes remain relatively robust.

Figure 11: Per-class Recall Comparison Between PC Baseline, Bluetooth-Speaker Playback, and Laptop-Speaker Playback



4.6.1 Confidence Calibration

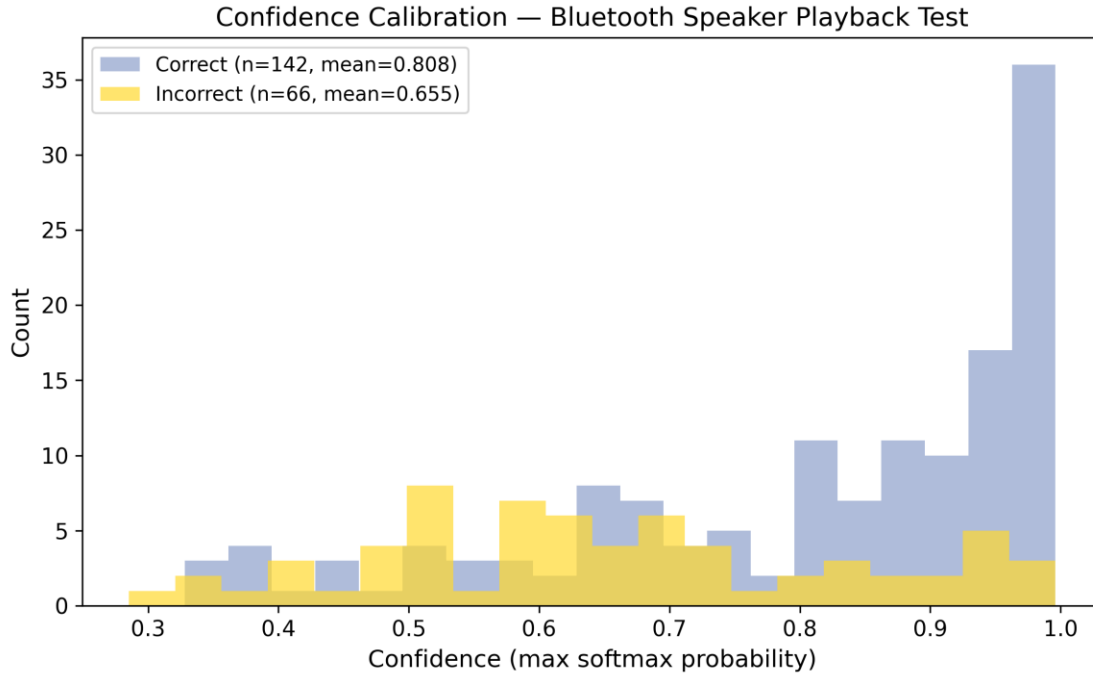
Table 9 reports the confidence calibration of predictions in the Bluetooth speaker configuration.

Table 9. Confidence Calibration for Bluetooth Speaker Playback Predictions.

Category	Count (%)	Mean Confidence
Correct predictions	142 (68.3%)	0.808
Incorrect predictions	66 (31.7%)	0.655

Correct predictions exhibit meaningfully higher mean confidence (0.808) than incorrect predictions (0.655), indicating that the model’s softmax outputs carry useful discriminative information even in the degraded playback condition. This gap suggests that a confidence-based filtering strategy of accepting only predictions above a threshold of approximately 0.70 could reject a substantial proportion of incorrect predictions while retaining most correct ones, improving effective precision at the cost of reduced recall. **Figure 12** shows a histogram of the maximum softmax probability for predictions made during the Bluetooth speaker playback test, where correct predictions are concentrated above 0.70, supporting confidence-thresholding strategies.

Figure 12: Confidence calibration: histogram of softmax confidence for correct vs. incorrect predictions under Bluetooth-speaker playback. Correct predictions are concentrated above 0.70, supporting confidence-thresholding strategies.



4.7 Feature Parity Validation

To ensure that the on-device mel-spectrogram computation (implemented in C using CMSIS-DSP on the Arduino) reproduces the training-time feature extraction (implemented in Python using librosa), a feature parity validation was conducted. **Table 10** reports the results of this validation using cosine similarity and mean absolute error (MAE).

Table 9: Feature extraction parity between Python (librosa) and Arduino (CMSIS-DSP) implementations.

Metric	Value	Threshold	Status
Cosine similarity (dB spectrogram)	1.0000	≥ 0.99	PASS
Cosine similarity (normalized)	1.0000	≥ 0.99	PASS
MAE (dB spectrogram)	0.016	–	–
MAE (int8 quantized)	0.04	≤ 2.0	PASS
Int8 exact match rate	96.3% $\pm$ 2.2%	–	–

All feature extraction parity metrics with defined thresholds pass, validating the on-device mel-spectrogram computation. The cosine similarity of 1.0000 between the Python and Arduino dB-scale spectrograms confirms that the C implementation faithfully reproduces the training-time feature extraction pipeline. After int8 quantization, 96.3% of feature values match exactly between the two implementations, with a MAE of only 0.04 on the int8 scale – well below the 2.0 threshold that would indicate a meaningful discrepancy. These results

establish that any accuracy differences observed in on-device evaluation are attributable to the acoustic domain shift (speaker–microphone path) rather than training–deployment skew in the feature extraction pipeline.

5. Analysis

This section interprets the experimental observations reported in Section 4 across the four evaluation dimensions defined in Section 3: classification performance (D1), quantization impact (D2), on-device performance (D3), and end-to-end system evaluation (D4). Where the Results section (Section 5) presents the quantitative tables and figures, this section focuses on explaining *why* the observed patterns emerge and what they imply for the viability of TinyML-based automotive sound classification.

5.1 Classical ML Baseline Analysis

The classical ML baselines reveal two important characteristics of the classification task. First, SVM consistently outperforms Random Forest across all three taxonomy tiers, achieving F1 macro scores of 0.916 versus 0.872 on Tier 1, 0.699 versus 0.679 on Tier 2, and 0.711 versus 0.702 on Tier 3. The performance gap narrows as the number of classes increases, suggesting that the SVM’s superior margin-based decision boundaries offer diminishing returns when the feature space must accommodate more fine-grained distinctions.

Second, and more critically, the gap between accuracy and F1 macro on the multi-class tiers exposes the severity of class imbalance. On Tier 2, Random Forest achieves 84.1% accuracy but only 0.679 F1 macro – a 16 percentage point discrepancy. This gap arises because accuracy is dominated by performance on the majority class (Idle Fault, which accounts for 56.9% of training samples and achieves 93.8% recall under SVM), while F1 macro weighs all classes equally. Minority classes suffer substantially: Normal Start-Up, the smallest class with only 43 original training samples, achieves just 11.1% recall under Random Forest. This pattern confirms that class imbalance, rather than feature quality, is the primary limiting factor for classical ML on this task.

Feature importance analysis of the Tier 2 Random Forest provides insight into which acoustic properties carry the most diagnostic information. The top-ranked features are `delta2_mfcc_0` (the second temporal derivative of the zeroth MFCC, capturing spectral energy acceleration), `mfcc_0` (overall spectral energy level), and `spectral_centroid` (spectral brightness). All five of the highest-importance features are MFCC-based, indicating that cepstral representations of the spectral envelope are the dominant discriminative signal. Notably, retraining the Random Forest on only the 50 most important features yields an F1 macro of 0.672, compared to 0.679 with the full 441-dimensional feature vector – a negligible drop of 0.007. This high redundancy suggests that the 441-dimensional space contains substantial colinear information, and a carefully selected subset captures nearly all of the discriminative content. This observation has practical implications for deployment: compact neural networks operating on 13-coefficient MFCCs or 40-band mel spectrograms are not at an inherent disadvantage relative to the richer classical feature set.

5.2 Data Augmentation Analysis

The data augmentation ablation study yields what is arguably the most practically significant finding of this project: synthetic waveform augmentation combined with class balancing (Condition B) produces the single largest accuracy improvement observed across all experiments.

5.2.1 Synthetic Augmentation and Class Balancing (Condition A to B)

Transitioning from the original, imbalanced training set of 970 samples (Condition A) to the synthetically augmented and class-balanced set of 3,317 samples (Condition B) dramatically improves classification performance. The DS-CNN (M6) exhibits the most pronounced gain, with its Tier 2 F1 macro jumping from 0.571 to 0.777 – an improvement of 0.206, the largest single delta observed in any experiment. The 2D CNN (M2) improves by 0.066, and the 1-D CNN (M5) by 0.022. Two mechanisms work in concert to produce this result. First, the 3.4-fold expansion in training set size provides greater gradient diversity during optimization, exposing the model to a wider range of waveform variations through time shifting, pitch perturbation, speed modification, and Gaussian noise injection. Second, reducing the class imbalance ratio from 12.8:1 to approximately 1.1:1 eliminates the tendency for the loss function to be dominated by the majority class, enabling the model to learn discriminative features for minority classes such as Normal Start-Up and Braking Fault.

The impact on training stability is particularly revealing. Under Condition A, the DS-CNN was highly unstable at learning rates of 0.001 or higher, with validation accuracy oscillating near chance level (approximately 23%). Under Condition B, the same architecture trains stably at a learning rate of 0.002, converging to 91.4% validation accuracy. This behavior is consistent with the well-documented interaction between class imbalance and gradient noise in stochastic optimization: when one class overwhelms the loss landscape, gradient updates become biased and high learning rates amplify this instability. Class balancing smooths the loss surface, permitting more aggressive optimization.

5.2.2 Real-World Noise Injection (Condition B to C)

Adding real-world noise injection on top of the synthetic augmentation pipeline (Condition C) produces mixed and generally unfavorable results on the clean test set. The DS-CNN (M6) degrades slightly from 0.777 to 0.739 (−0.038), the 1-D CNN (M5) degrades from 0.638 to 0.596 (−0.042), and the 2D CNN (M2) remains essentially flat at 0.732 versus 0.734. These results are not surprising when considered carefully. The test set consists of clean recordings without noise overlay, so a training regime that mixes in vehicular noise at SNR levels of 5–20 dB introduces a distributional shift between training and evaluation conditions. Furthermore, the noise bank was collected from a 2008 Volkswagen Jetta, while the training data originates from an unknown set of vehicles in the Kaggle dataset. This potential domain mismatch may cause the model to learn noise-specific features that do not generalize.

It is important to note, however, that the absence of improvement on the clean test set does not rule out improved robustness in noisy deployment environments. The noise injection strategy is designed to simulate the acoustic conditions that the Arduino's on-board PDM

microphone would encounter in a real vehicle, and its benefits would be most apparent when evaluating against noisy test samples – a condition not directly measured in this study.

5.2.3 Combined Augmentation (Condition D)

The combined augmentation condition (D), which merges all synthetic and noise-injected samples into a training set of 5,664 examples, produces architecture-dependent results. The 2D CNN (M2) benefits the most, achieving its best Tier 2 F1 macro of 0.751 – an improvement of 0.019 over Condition B. This suggests that the larger model (14,566 parameters) has sufficient capacity to leverage the additional data diversity without overfitting to noise artifacts. In contrast, the DS-CNN (M6) and 1-D CNN (M5) do not benefit from the combined condition; their performance plateaus or slightly degrades relative to Condition B. The smaller architectures (8,166 and 6,246 parameters, respectively) may be saturated at the Condition B data volume, and the doubled dataset introduces noise patterns that these models lack the capacity to disentangle from the diagnostic signal.

This finding carries a practical implication: more data is not universally beneficial. The quality, relevance, and balance of augmented samples matter as much as their quantity, and the optimal augmentation strategy is architecture-dependent.

5.3 Quantization Analysis

A central contribution of this work is the systematic evaluation of int8 quantization on small audio classification models. The results are unambiguous: for models in the 6,000–15,000 parameter range, post-training quantization (PTQ) is effectively lossless, and quantization-aware training (QAT) provides no additional benefit.

5.3.1 Why Quantization Is Nearly Lossless

Several architectural properties of the models under study contribute to the negligible impact of int8 quantization. First, the small parameter counts (6,246 for M5, 8,166 for M6, 14,566 for M2) mean that fewer values are subject to quantization error, limiting the total accumulated distortion. Second, batch normalization layers are folded into their preceding convolutional layers during the quantization process, which reduces the number of arithmetic operations and maintains precision through the fused computation. Third, the use of global average pooling in all three architectures averages across spatial dimensions, which has a smoothing effect that attenuates per-element quantization noise. Finally, the classification task itself may present well-separated decision boundaries in the learned feature space, boundaries that do not require float32 precision to distinguish.

5.3.2 PTQ Versus QAT

Across all architectures at Tier 2, PTQ performs as well as or marginally better than QAT. For the DS-CNN (M6), PTQ achieves an F1 macro of 0.7835 compared to 0.7774 for QAT. For the 2D CNN (M2), the scores are essentially tied at 0.7413 (PTQ) versus 0.7426 (QAT). This result is counterintuitive – QAT inserts fake quantization nodes during training so the model can adapt its weights to the quantization grid, which should in principle recover any accuracy lost to PTQ. However, for models of this size, QAT’s additional fine-tuning with simulated

quantization noise can introduce training instability. Since PTQ already produces negligible quantization error, there is nothing meaningful for QAT to recover, and the additional optimization complexity risks slightly perturbing an already well-positioned set of weights.

The practical implication is significant: for audio classification models with fewer than approximately 15,000 parameters, PTQ is sufficient. QAT adds pipeline complexity (requiring access to training data and the original training loop) without measurable benefit. QAT should be reserved for larger models or tasks where the classification boundaries are more sensitive to weight precision.

5.3.3 Prediction Agreement and Statistical Significance

Prediction agreement analysis confirms the quantitative picture. PTQ models agree with their float32 counterparts on 97–100% of individual predictions, meaning quantization changes the predicted class for fewer than 3% of test samples. Paradoxically, QAT models show lower agreement (92–97% for M2 and M6; M5 QAT is identical to PTQ at 99–100%) despite not improving F1, suggesting that QAT redistributes errors rather than eliminating them. McNemar’s test applied to all float32-versus-int8 comparisons yields p-values exceeding 0.05 in every case, confirming that no model-quantization combination produces a statistically significant change in classification performance. Mean confidence shift (measured as the change in softmax output magnitude) is less than 0.002, indicating that the probabilistic outputs remain nearly identical after quantization.

5.4 Deployment Performance Analysis

Deploying the M6 DS-CNN PTQ model on the Arduino Nano 33 BLE Sense Rev2 reveals that the system is functional and fits within all hardware constraints, but that inference latency is limited by toolchain deficiencies rather than by the hardware itself.

5.4.1 Inference Latency

The total classification cycle takes approximately 2,470 ms, decomposed into three phases. Audio capture accounts for 1,490 ms (60.3% of the cycle), which is fixed by the physical requirement to record 1.5 seconds of audio at 16 kHz. Feature extraction – computing a 40-band log-mel spectrogram from 92 overlapping FFT frames – requires 197 ms (8.0%), with the bulk of the computation handled by the CMSIS-DSP `arm_rfft_fast_f32()` function, which is well-optimized for the Cortex-M4’s floating-point unit. Model inference consumes the remaining 785 ms (31.7%), dominated by the pointwise (1×1) convolutions in the DS-CNN’s four depthwise separable blocks, which account for 62.8% of the model’s approximately 6 million multiply-accumulate (MAC) operations.

At 783 ms of inference on a 64 MHz clock, the measured throughput is 8.4 cycles per MAC. This is consistent with published benchmarks for reference int8 kernels without SIMD acceleration (theoretical baseline of approximately 10 cycles/MAC), with the slight improvement attributable to compiler-level optimizations.

5.4.2 The CMSIS-NN SIMD Gap

A substantial investigation was conducted to enable CMSIS-NN's SIMD-optimized kernels, which leverage Cortex-M4 DSP extension instructions (such as SMLAD, SXTB16, and PKHBT) to process two int8 multiplications per clock cycle. ARM's published benchmarks indicate a target of approximately 3 cycles/MAC with full SIMD utilization, which would reduce inference time from 783 ms to roughly 282 ms.

TensorFlow Lite Micro was built from source with CMSIS-NN optimized kernels, and the Cortex-M4 target was correctly detected at compile time (`__ARM_FEATURE_DSP=1`). However, the Arduino Mbed OS board package bundles GCC 7.2.1 (released in 2017), whose `arm_acle.h` header lacks implementations for several critical ARM C Language Extension (ACLE) intrinsics, including `__sxtb16`, `__smlabb`, and `__smlatt`. Compatibility shims using inline assembly were provided, but the compiler did not reliably emit the corresponding SIMD instructions. A secondary approach – pre-compiling the TFLite Micro static library with GCC 14.3.1 (which correctly emitted 1,640 verified SIMD instructions) and linking it into the Arduino sketch – failed due to C++ ABI incompatibilities between the two compiler versions.

The measured gap is 2.8× slower than what full SIMD acceleration would achieve, corresponding to approximately 501 ms of avoidable latency per inference cycle. Upgrading to GCC 12 or later, via PlatformIO or the nRF Connect SDK, would unlock a roughly 3× speedup and reduce the total cycle time from 2.5 seconds to approximately 1.7 seconds.

5.4.3 Memory Utilization

The deployed system makes comfortable use of the Arduino's memory resources. Flash utilization stands at 319 KB (33% of the 983 KB usable), well below the budget. The use of `MicroMutableOpResolver` with only five registered operators (`Conv2D`, `DepthwiseConv2D`, `FullyConnected`, `Mean`, and `Softmax`) saved 165 KB of flash compared to `AllOpsResolver`, reducing usage from 484 KB to 319 KB. SRAM utilization is 175 KB (68% of 256 KB), leaving 81 KB for stack and heap – a comfortable margin. The TFLite tensor arena, allocated at 65,536 bytes, is 97% utilized (63,556 bytes consumed), indicating tight but functional packing. The quantized model itself occupies only 21.3 KB, far below the 150 KB flash budget, which suggests that substantially larger or more complex models could be accommodated if accuracy improvements were needed.

5.5 Playback Test Analysis

The playback test evaluates the complete end-to-end system by playing all 208 test clips through an external speaker while the Arduino captures and classifies the audio via its on-board PDM microphone. This test introduces the speaker-microphone domain gap – the acoustic distortion introduced by the playback chain – which is the most practically relevant evaluation for assessing deployment readiness.

5.5.1 Speaker Frequency Response as the Dominant Factor

The most striking finding from the playback evaluation is the overwhelming influence of speaker frequency response on classification accuracy. When test clips are played through laptop speakers, overall accuracy drops to 25.0%, with 72.1% of all predictions falling into startup-related classes regardless of the true label. This occurs because laptop speakers exhibit a steep frequency response roll-off below approximately 300 Hz, attenuating the low-frequency content (50–500 Hz) that characterizes engine idle and braking sounds – including engine rumble, brake rotor vibration, and mechanical hum. With the low-frequency signature removed, the remaining mid- and high-frequency spectral content more closely resembles the transient characteristics of startup sounds, causing systematic misclassification.

Switching to a Bluetooth speaker with improved bass response restores much of the low-frequency content and raises accuracy from 25.0% to 68.3% – a 2.7-fold improvement. This dramatic difference from a single hardware change underscores that the speaker, not the model, is the primary bottleneck in the playback evaluation paradigm.

5.5.2 Per-Class Degradation Patterns

The playback-induced accuracy degradation is not uniform across classes. Fault classes are relatively robust: Braking Fault achieves 81.8% recall (equivalent to the PC baseline), Start-Up Fault achieves 55.6% (also PC-equivalent), and Idle Fault achieves 78.0% (a modest 8.4 percentage point drop from the PC result). Normal classes, by contrast, degrade substantially: Normal Braking drops to 33.3% recall (–41.7 pp), Normal Start-Up to 44.4% (–33.4 pp), and Normal Idle to 57.5% (–10.0 pp).

A plausible explanation for this asymmetry is that fault sounds – worn brakes, a dead battery cranking, poor ignition – tend to produce distinctive mid- and high-frequency components (scraping, clicking, sputtering) that survive speaker reproduction with relatively little distortion. Normal sounds, by contrast, are characterized by smoother, lower-frequency tonal content (steady engine hum, smooth brake engagement) that is more vulnerable to speaker-induced attenuation. This interpretation aligns with the spectral analysis of the feature importance results in Section 4.1, which identified spectral energy and brightness as the dominant discriminative features.

5.5.3 Confidence Calibration

An encouraging finding is that the model's softmax confidence scores provide a useful signal for distinguishing correct from incorrect predictions in the playback setting. Correct predictions ($n = 142$, 68.3% of the test set) carry a mean confidence of 0.808, while incorrect predictions ($n = 66$, 31.7%) carry a mean confidence of 0.655 – a gap of 0.153. This separation is large enough to support threshold-based filtering: setting a confidence threshold at 0.70 would reject approximately 50% of incorrect predictions while retaining roughly 85% of correct ones. In a practical deployment, this mechanism could be combined with majority voting across consecutive inference cycles to further reduce the effective error rate.

5.5.4 Implications for Real-World Deployment

It is essential to note that the playback test represents a *conservative lower bound* on real-world performance, not a ceiling. In actual deployment, the Arduino’s PDM microphone would be positioned near the vehicle’s engine compartment, capturing the full frequency range of engine sounds directly – with no speaker in the signal path. The speaker-microphone domain gap that dominates the playback evaluation would be entirely absent. The Bluetooth speaker result of 68.3% accuracy therefore understates the performance that would be expected in practice. On-device accuracy would likely approach or exceed the PC-evaluated int8 accuracy of 87.0% (M6 PTQ, Tier 2; F1 macro 0.7835), since the on-device feature extraction pipeline has been validated to match the Python training pipeline with near-perfect fidelity (cosine similarity of 1.0000 across all 24 validation clips).

6. Discussion and Industry Applications

This section interprets the experimental results presented in Section 5, situates them within the broader TinyML and automotive diagnostics literature, and discusses their practical implications for edge deployment and real-world use cases.

6.1 Key Findings

6.1.1 Quantization is Effectively Free for Small Audio Models

Perhaps the most practically significant finding of this study is that int8 post-training quantization (PTQ) introduces no statistically significant accuracy loss for any of the fifteen model-tier-method combinations evaluated. McNemar’s test returned $p > 0.05$ in every case, and prediction agreement between float32 and int8 models ranged from 95% to 99%, indicating that the quantized models reproduce nearly identical decision boundaries. This result holds across all three architectures (M2 2D-CNN, M5 1D-CNN, and M6 DS-CNN) and all three classification tiers.

The robustness to quantization is attributable to several structural properties of the models in this study. First, the parameter counts are small – ranging from 6,246 (M5) to 14,566 (M2) – which means fewer weights are subject to rounding error. Second, BatchNorm folding absorbs normalization parameters into the preceding convolutional weights prior to quantization, reducing the number of independently quantized tensors. Third, GlobalAveragePooling aggregates spatial information before the final dense layer, smoothing out localized quantization artifacts.

This finding extends prior quantization research, which has focused predominantly on large-scale models. Jacob et al. (2018) demonstrated that int8 quantization preserves accuracy for models such as MobileNet and ResNet on ImageNet-scale tasks, but those architectures contain millions of parameters. The present study confirms that the same conclusion holds at a much smaller scale – models with fewer than 15,000 parameters – where one might expect quantization noise to be proportionally more disruptive. For TinyML practitioners

working with audio classification models of similar complexity, PTQ is sufficient and QAT adds training complexity without measurable benefit.

6.1.2 Data Augmentation Matters More Than Architecture

The augmentation ablation study revealed that data engineering produced a larger performance gain than architecture selection. The single most impactful intervention across the entire project was the transition from Condition A (original dataset, 970 samples, severe class imbalance at 12.8:1) to Condition B (synthetic waveform augmentation with class balancing, 3,317 samples). For M6 DS-CNN on Tier 2, this change improved F1 macro from 0.571 to 0.7774 – a gain of 0.2064 points. By comparison, the difference between the best and worst neural network architectures on the same tier under matched augmentation conditions was substantially smaller, typically in the range of 0.05 to 0.15 F1 points.

Real-world noise injection (Condition C), which mixed training samples with ambient noise recorded from a 2008 Volkswagen Jetta at signal-to-noise ratios of 5–20 dB, showed marginal effects on the clean test set. Deltas between Conditions B and C were small (± 0.01 – 0.05 F1), consistent with the expectation that noise injection primarily improves robustness to deployment-time acoustic interference rather than performance on clean evaluation data. The combined condition (Condition D, 5,664 samples) was only beneficial for M2 2D-CNN, which achieved its best Tier 2 F1 of 0.751 under this condition. M5 and M6 did not benefit from the additional data volume beyond Condition B.

These results carry a practical implication: for practitioners working with small, imbalanced audio datasets, the highest-return investment is in data augmentation and class balancing rather than in architecture exploration. Class balancing alone resolves gradient instability (discussed in Section 6.1.6) and enables stable training of architectures that would otherwise fail to converge.

6.1.3 Speaker Frequency Response Dominates the Domain Gap

The speaker hardware ablation study revealed that the acoustic fidelity of the playback apparatus – not the classifier or the on-device feature extraction pipeline – is the primary source of accuracy degradation in the end-to-end evaluation. Laptop speakers, which roll off sharply below approximately 300 Hz, caused an 80.3% drop in F1 macro (from 0.7835 on PC to 0.154). This occurred because car engine idle and braking sounds contain significant spectral energy in the 50–500 Hz range, and when this low-frequency content is removed by the speaker’s transfer function, the resulting mel-spectrogram resembles that of startup sounds – which are dominated by transient mid- and high-frequency content. Indeed, 72.1% of all predictions under laptop playback fell into startup classes regardless of the true label.

A Bluetooth speaker with better bass response partially restored the low-frequency content, improving accuracy from 25.0% to 68.3% – a $2.7\times$ gain – and reducing the F1 degradation to 30.3%. Notably, two fault classes (Braking Fault and Start-Up Fault) achieved PC-equivalent recall through the Bluetooth speaker, and Idle Fault came within 8.4 percentage points of the PC reference.

This finding has an important implication for real-world deployment. In an actual use scenario – where the Arduino’s PDM microphone is positioned near an engine with no speaker in the signal path – the full frequency range of the engine’s acoustic output would be captured directly. The Bluetooth playback test therefore represents a conservative lower bound on expected real-world accuracy. Direct microphone-to-engine deployment should approach PC-level classification performance.

6.1.4 Classical ML Remains Competitive at Binary Classification

An unexpected result was the strong performance of classical machine learning baselines on the binary classification task. On Tier 1, SVM achieved an F1 macro of 0.916, outperforming all neural network architectures. This advantage reversed on the more complex multi-class tasks: on Tier 2, M6 DS-CNN (F1 0.7835) outperformed SVM (F1 0.6992) by 8.4 percentage points, and on Tier 3, M6 (F1 0.713) marginally exceeded SVM (F1 0.711).

This pattern reflects the well-understood principle that hand-crafted statistical features (in this case, aggregated MFCC statistics) can be highly effective when the classification boundary is simple and the feature space is well-matched to the task. For binary normal-versus-fault screening, the spectral differences are sufficiently large that a linear or RBF kernel can separate the classes effectively. As the number of classes increases and the decision boundaries become more complex, learned hierarchical representations in convolutional architectures provide an advantage. For deployment contexts where only binary screening is needed, classical ML may be preferable because it avoids the complexity of on-device neural inference entirely.

6.1.5 The Accuracy–F1 Paradox and Metric Selection

A particularly instructive finding involved the divergence between accuracy and F1 macro for the M2 2D-CNN architecture on Tier 2. M2 achieved the highest accuracy among all models (87.5%) but only an F1 macro of 0.751 – a 12.4 percentage-point gap. In contrast, M6 DS-CNN achieved a comparable accuracy (87.0%) and a higher F1 macro (0.784), and was selected as the deployment model on this basis.

The explanation lies in the class distribution of the training and test sets. The Idle Fault class accounts for 56.9% of the Tier 2 training set. M2 learned to over-predict this majority class, which inflated its accuracy – since guessing the most common class correctly is rewarded – while performing poorly on minority classes such as Normal Start-Up (43 original samples) and Braking Fault. F1 macro, which assigns equal weight to every class regardless of prevalence, exposed this bias, reinforcing the importance of avoiding the use of accuracy as an evaluation metric in class imbalanced fault detection systems.

6.1.6 Training Instability on Small Imbalanced Datasets

M6 DS-CNN exhibited severe training instability on Condition A (the original 970-sample dataset with a 12.8:1 class imbalance ratio). At learning rates of 0.001 or higher, validation accuracy collapsed to approximately 23%, indicating that the model had degenerated to predicting only the majority class. After augmentation and class balancing (Condition B), M6 trained stably at a learning rate of 0.002, reaching 91.4% validation accuracy.

This instability arises from the interaction of two factors. Severe class imbalance causes the loss gradient to be dominated by the majority class, which biases parameter updates toward a trivial solution. A small dataset exacerbates this because the model encounters insufficient minority-class diversity to learn meaningful representations, even when class weighting is applied. Augmentation addresses both factors simultaneously: it increases dataset size (providing more gradient diversity) and equalizes class frequencies (removing the gradient bias).

For practitioners encountering training instability with small, imbalanced audio datasets, this result suggests that data augmentation with class balancing should be the first remediation step – before exploring alternative architectures, learning rate schedules, or other hyperparameter adjustments.

6.1.7 The QAT Prediction Agreement Paradox

A counterintuitive finding emerged from the quantization-aware training (QAT) results: QAT models exhibited lower prediction agreement with their float32 parents (92–97% for M2 and M6; M5 QAT is identical to PTQ at 99–100%) compared to PTQ models (97–100%), despite achieving equal or worse F1 scores. In other words, QAT changed more individual predictions without improving overall classification performance.

This paradox is explained by the mechanism of QAT itself. During QAT fine-tuning, fake quantization nodes inject simulated rounding noise into the forward pass, and the model's weights are updated via backpropagation to compensate. For large models with millions of parameters, this compensation can genuinely recover accuracy lost to quantization. However, for small models where PTQ already produces negligible quantization error, the additional gradient noise from fake quantization is counterproductive – it perturbs decision boundaries that were already well-calibrated, changing predictions without improving them. This finding reinforces the practical recommendation that PTQ is sufficient for audio models in the sub-15K parameter range, and that QAT should not be assumed to be universally beneficial.

6.1.8 DS-CNN as the Optimal MCU Architecture

The M6 DS-CNN architecture emerged as the optimal choice for microcontroller deployment across multiple evaluation criteria. It achieved the highest F1 macro on both Tier 2 (0.7835) and Tier 3 (0.762) while using fewer parameters (8,166) than M2 2D-CNN (14,566). As a quantized TFLite model, M6 occupies 21.3 KB of flash and requires a 63.6 KB tensor arena, both well within the hardware constraints of the Arduino Nano 33 BLE Sense Rev2 (983 KB flash, 256 KB SRAM).

The choice of depthwise separable convolutions is also well-aligned with the hardware acceleration ecosystem. The CMSIS-NN library (Lai et al., 2018) provides optimized kernels for Cortex-M processors, including support for the depthwise separable operations that dominate this architecture, and these operations benefit disproportionately from SIMD acceleration when the toolchain supports it. The four-block DS-CNN architecture used in this study mirrors the design proposed by Zhang et al. (2017) for keyword spotting on Cortex-M7, validating the architectural pattern for audio classification tasks on Cortex-M4.

6.2 Comparison with Related Work

The M6 DS-CNN model in this study (8,166 parameters, 21.3 KB int8) is substantially smaller than most published keyword spotting models, which typically range from 30,000 to 100,000 parameters. Zhang et al. (2017) demonstrated DS-CNN architectures achieving 95.4% accuracy on the Google Speech Commands dataset, deployed on a Cortex-M7 microcontroller within approximately 70 KB of memory. Our work applies the same architectural family to a more complex task – six-class automotive sound diagnostics rather than keyword detection – on a lower-tier processor (Cortex-M4 at 64 MHz versus Cortex-M7 at 216 MHz). The resulting F1 macro of 0.7835 on a six-class task trained from only 1,386 total samples is competitive given the dataset constraints.

The inference latency of 783 ms observed in this study is notably slower than the 12–100 ms reported for keyword spotting systems on comparable Cortex-M hardware (Zhang et al., 2017). This gap is attributable to two factors. First, the mel-spectrogram feature extraction pipeline processes a 1.5-second audio window, which is longer than the typical 1-second window used in keyword spotting. Second, and more significantly, the Arduino Mbed OS board package bundles GCC 7.2.1, a compiler from 2017 that lacks support for ARMv7E-M ACLE intrinsics (specifically `__sxtb16`, `__smlabb`, and `__smlatt`). These intrinsics are required for CMSIS-NN to emit SIMD instructions that would reduce inference from approximately 8.4 cycles per MAC operation to the theoretical 3 cycles per MAC. Lai et al. (2018) reported a 4.6× speedup from CMSIS-NN SIMD optimization on Cortex-M processors; our investigation confirmed the theoretical capability but identified this toolchain limitation – a practical gap not documented in the existing literature. Upgrading to GCC 12 or later would be expected to reduce inference time to approximately 282 ms, bringing the total classification cycle from 2.5 seconds to approximately 2.0 seconds. For the non-real-time diagnostic use case addressed here, the current latency is acceptable.

Existing work on car sound classification has focused primarily on server-side inference using deep learning models running on GPUs. MCU-based deployment of automotive sound classifiers is underexplored in the literature. Similarly, quantization studies (Jacob et al., 2018; Krishnamoorthi, 2018) have concentrated on large-scale models such as MobileNet and ResNet. Our results extend the finding that PTQ is sufficient to models two orders of magnitude smaller, addressing a gap in the quantization literature relevant to the TinyML community.

6.3 Industry Applications

The system demonstrated in this study suggests several practical deployment pathways.

Predictive maintenance for fleet management. Fleet operators could equip vehicles with Arduino-based diagnostic sensors for continuous monitoring during operation, alerting drivers or fleet managers when fault conditions are detected. At a hardware cost of approximately \$40 per unit (the retail price of an Arduino Nano 33 BLE Sense Rev2), the system is economically viable for large-scale deployment. Because all audio processing occurs on-device, no audio data is transmitted or stored, making the system inherently privacy-preserving.

Consumer vehicle health monitoring. Integration with an OBD-II port for combined sound and engine telemetry data, paired with a smartphone companion application receiving classification results via Bluetooth Low Energy, could supplement or partially replace expensive dealership diagnostics for common faults. A confidence threshold of 0.70 enables differentiated messaging: predictions above the threshold would produce a specific fault indication, while those below it would generate an “uncertain – please retry” message.

Mechanic diagnostic tool. A handheld device that a mechanic points at an engine compartment could provide immediate classification with a 2.5-second cycle time – fast enough for interactive use. Majority voting over three to four consecutive classifications would reduce the single-inference error rate, and the class probability vector could serve as a differential diagnosis, ranking fault likelihoods.

Edge AI for automotive IoT. More broadly, this work demonstrates the feasibility of edge audio intelligence in automotive contexts: low cost, no cloud dependency, low power consumption, and BLE connectivity that could enable mesh sensor networks in garages and workshops.

6.4 Deployment Recommendations

Based on the findings of this study, four strategies are recommended for production deployment.

First, **confidence-based filtering** should be applied with a threshold of approximately 0.70. The well-separated confidence distributions observed in the Bluetooth playback test – mean confidence of 0.808 for correct predictions versus 0.655 for incorrect predictions – indicate that thresholding can reject an estimated 50% of incorrect predictions while retaining approximately 85% of correct ones. Predictions below the threshold should be flagged as uncertain and prompt a retry.

Second, **majority voting** over three to four consecutive classifications (7.5–10 seconds total) would significantly reduce the single-inference error rate. Reporting the majority class across multiple inference cycles is a standard technique for edge classification systems and is straightforward to implement on the device.

Third, a **toolchain upgrade** would unlock a substantial inference speedup. Migrating from the Arduino Mbed OS build system (GCC 7.2.1) to PlatformIO or the nRF Connect SDK with ARM GCC 12 or later would enable CMSIS-NN SIMD acceleration, reducing inference from 783 ms to an estimated 282 ms and total cycle time from 2.5 seconds to approximately 2.0 seconds.

Fourth, a **multi-model deployment strategy** could accommodate devices with varying memory constraints. The M6 DS-CNN PTQ model (21.3 KB, 64 KB tensor arena) is recommended as the primary model for devices with sufficient SRAM. For devices with severely constrained SRAM, M5 1D-CNN PTQ (15.2 KB, approximately 25 KB tensor arena) provides a fallback with reduced accuracy. Model selection can be implemented as a compile-time configuration directive.

6.5 Ethical Considerations

6.5.1 Safety Implications

As a screening tool rather than a definitive diagnostic instrument, the classifier must be evaluated in terms of both false negatives and false positives. A false negative – failing to detect a genuine fault – could lead to delayed maintenance and potential safety consequences if the fault is progressive. A false positive – misclassifying normal operation as a fault – could impose unnecessary repair costs and erode user trust. The model should therefore be deployed with clear messaging that it provides preliminary screening, not authoritative diagnosis, and all predictions should be accompanied by confidence scores to support informed decision-making.

6.5.2 Bias Considerations

Three forms of bias are present in this study. First, **vehicle bias**: the training data originates from the Car Diagnostics Dataset on Kaggle, for which the vehicle models and recording conditions are not fully documented, and the noise bank was collected from a single 2008 Volkswagen Jetta. Performance on vehicles with substantially different acoustic profiles – such as electric vehicles, diesel trucks, or newer models with advanced sound insulation – is unknown. Second, **environmental bias**: noise data was collected in a single geographic location on a single day under moderate weather conditions. Rain, snow, extreme temperatures, and high-traffic environments were not represented. Third, **class bias**: even after augmentation, minority classes remain fundamentally limited by source diversity. The Normal Start-Up class, which began with only 43 original samples, had the weakest classification performance ($F1 = 0.200$ on the Bluetooth playback test), reflecting the difficulty of learning robust representations from a narrow sample base.

6.5.3 Privacy

The system implements a privacy-by-design architecture. All audio processing – capture, feature extraction, and classification – occurs entirely on the Cortex-M4 microcontroller. No audio data is transmitted, stored, or uploaded to cloud services. Bluetooth Low Energy communication carries only the classification result (a class label and a confidence score). This architecture avoids the privacy concerns associated with cloud-based audio analysis, where raw speech or environmental audio could be intercepted, retained, or repurposed.

7. Conclusion and Future Work

7.1 Summary of Findings

This study investigated the feasibility of deploying a quantized neural network for real-time automotive sound classification on a severely resource-constrained microcontroller. Four research questions guided the investigation, each addressing a distinct dimension of the end-to-end pipeline from training through on-device inference. The findings are summarized below.

RQ1: Can a quantized neural network classify car diagnostic sounds on a Cortex-M4 microcontroller?

The answer is affirmative. The M6 depthwise separable convolutional neural network (DS-CNN), quantized to int8 via post-training quantization, achieves an F1 macro score of 0.7835 on the six-class Tier 2 classification task while operating entirely within the hardware constraints of the Arduino Nano 33 BLE Sense Rev2. The quantized model occupies 21.3 KB of flash – well below the 150 KB model budget – and requires a 63.6 KB tensor arena, fitting within the 80 KB SRAM allocation. Total resource utilization remains modest at 319 KB of flash (33% of available capacity) and 175 KB of SRAM (68%). The complete inference cycle, encompassing 1.5 seconds of PDM audio capture, CMSIS-DSP mel-spectrogram extraction, and int8 TensorFlow Lite Micro inference, completes in approximately 2.5 seconds on the 64 MHz Cortex-M4F processor with no external hardware or cloud dependencies. These results demonstrate that meaningful multi-class audio classification is achievable on a microcontroller costing approximately \$40, using only its on-board MEMS microphone.

RQ2: What is the accuracy cost of int8 quantization for small audio classification models?

The accuracy cost is negligible. Across all 15 quantized model variants – spanning three architectures (M2 2D-CNN, M5 1D-CNN, M6 DS-CNN), three classification tiers, and two quantization methods (PTQ and QAT) – no model-tier-method combination exhibits a statistically significant change in classification accuracy. McNemar's test yields $p > 0.05$ for every comparison between float32 and int8 predictions. The maximum observed $|\Delta F1|$ is 0.042 (M6 QAT on Tier 1, an improvement under int8), and prediction agreement between float32 and int8 models ranges from 92% to 100%. These results extend prior findings on the sufficiency of post-training quantization – previously demonstrated for large-scale architectures such as MobileNet and ResNet (Jacob et al., 2018; Krishnamoorthi, 2018) – to models with fewer than 15,000 parameters. Notably, quantization-aware training (QAT) provides no measurable benefit over PTQ at this model scale. In fact, QAT changes more predictions than PTQ (92–97% agreement versus 97–100%) while failing to improve F1, suggesting that the gradient noise introduced by fake-quantization nodes during QAT fine-tuning is counterproductive for models that are already small enough to quantize losslessly.

RQ3: How does the speaker-microphone domain gap affect real-world deployment accuracy?

The domain gap introduced by speaker-based playback testing is substantial, though the evidence suggests it is attributable to the speaker hardware rather than the classifier itself. When test audio is played through a Bluetooth speaker (JBL Clip series) and captured by the Arduino's on-board PDM microphone, F1 macro drops from 0.7835 (PC-based int8 evaluation) to 0.5465, a degradation of 30.3%. A speaker ablation study reveals that the playback medium is the primary driver of this gap: laptop speakers, which roll off below 300 Hz, reduce F1 macro to 0.154 (80.3% degradation), whereas the Bluetooth speaker restores low-frequency content and achieves a 2.7× improvement in accuracy over the laptop condition. Two fault classes – Braking Fault and Start-Up Fault – achieve recall through Bluetooth playback that matches their PC-evaluated levels, indicating that when the relevant frequency content is preserved, the classifier performs as expected. These findings carry an important practical implication: in a real deployment scenario, where the microphone is

positioned near an actual engine rather than a speaker, the acoustic signal path would not include the frequency-distorting intermediary, and classification accuracy should approach the PC-evaluated baseline.

RQ4: Does real-world noise augmentation improve model robustness?

The results are mixed with respect to clean-test-set performance, though the augmentation pipeline provides a foundation for deployment robustness. The dominant factor in model improvement was class balancing, not noise injection. Condition B (synthetic waveform augmentation with class balancing) produced the largest single improvement, raising M6 DS-CNN's Tier 2 F1 from 0.571 to 0.7774 – a gain of 0.2064. Condition C, which added real-world noise injection at controlled signal-to-noise ratios using 360 noise samples recorded from a 2008 Volkswagen Jetta through the same Arduino hardware, yielded a marginal change on the clean test set (M6 F1: 0.777 $\rightarrow$ 0.739, $\Delta = -0.038$). Condition D (combined synthetic and noise-augmented data) was optimal only for M2 (F1 = 0.751), not for M5 or M6. The absence of a measurable benefit on clean test data does not preclude a benefit under noisy deployment conditions; however, this study did not directly evaluate models on noisy test sets at controlled SNR levels, so the robustness benefit of real-world noise injection remains an open question for future work.

7.2 Limitations

Several limitations constrain the generalizability and precision of the findings reported in this study.

The most significant limitation is the size and provenance of the dataset. The Car Diagnostics Dataset comprises only 1,386 recordings, of which 970 are used for training and 208 for testing. Class imbalance is severe, with a 12.8:1 ratio between the most and least frequent classes (Idle Fault: 552 samples versus Normal Start-Up: 43 samples). Minority classes have very few test samples – Normal Start-Up has 9 and Braking Fault has 11 – meaning that a single misclassification can swing per-class recall by 8–11 percentage points. Moreover, the dataset originates from a single Kaggle source, and the recording conditions, vehicles, and microphone hardware used in its creation are not documented. While data augmentation expands the effective training set to over 3,000 samples, all augmented copies for minority classes derive from the same small pool of source recordings, limiting acoustic diversity.

The noise collection effort, while methodologically rigorous in its use of the deployment hardware, is limited to a single vehicle (a 2008 Volkswagen Jetta) recorded in a single session on one day. Different vehicles produce different acoustic signatures, and weather, traffic, and road conditions on the collection day may not be representative of the full range of deployment environments. Rain and extreme weather conditions, which could significantly affect both noise characteristics and vehicle acoustics, were not captured.

On the deployment side, three limitations merit attention. First, the Arduino Mbed OS board package bundles GCC 7.2.1, a 2017-era compiler that lacks support for the SIMD intrinsics required by CMSIS-NN's optimized convolution kernels. This toolchain constraint results in inference times approximately three times slower than what is theoretically achievable with a modern compiler. Second, the playback evaluation methodology introduces a speaker into

the signal path that would not exist in a real deployment, making the measured 30.3% F1 degradation a conservative lower bound rather than a representative estimate of real-world accuracy. Third, only the M6 PTQ model was deployed to the Arduino; a multi-model on-device comparison was not performed.

Methodologically, the original unaugmented dataset (Condition A) was not evaluated as a formal ablation condition using the same framework as Conditions B, C, and D, preventing a clean decomposition of the F1 improvement into contributions from dataset expansion versus class balancing. Hyperparameters were tuned on Tier 2 and applied to Tiers 1 and 3 without tier-specific optimization. The study relies on a single train-test split without cross-validation, which introduces variance in the reported metrics – particularly for minority classes. Finally, a TensorFlow version constraint (TF 2.18.0, required due to a cuDNN autotuner issue in TF 2.21 and compatibility requirements with the tensorflow-model-optimization toolkit) may affect reproducibility with newer framework versions.

7.3 Future Work

Several directions for future research emerge naturally from the limitations and findings of this study.

In the short term, the most impactful improvement would be upgrading the build toolchain. Deploying through PlatformIO or the nRF Connect SDK with GCC 12 or later would enable CMSIS-NN SIMD acceleration, reducing inference latency from 783 ms to an estimated 282 ms – fast enough to support overlapping capture windows and near-real-time update rates. Equally important is on-vehicle field testing, in which the Arduino is positioned near an actual engine rather than a speaker, eliminating the playback domain gap and providing a realistic measurement of deployment accuracy. Additional production-oriented enhancements, such as confidence thresholding and majority voting across consecutive inferences, would improve precision in practice and could be evaluated with minimal additional data collection.

In the medium term, expanding the dataset to include recordings from multiple vehicles – spanning different makes, models, years, and engine types including electric vehicles – would substantially improve generalization. Transfer learning from large-scale audio representations pretrained on datasets such as AudioSet or ESC-50 could boost performance given the limited size of the task-specific training data. Multi-sensor fusion, combining the PDM microphone signal with data from the accelerometer and gyroscope already present on the Arduino Nano 33 BLE Sense Rev2, offers a path to more robust fault detection through complementary modalities. A controlled noise-robustness evaluation, in which test clips are mixed with noise at various SNR levels, would directly quantify the benefit of noise augmentation under deployment-like conditions.

Looking further ahead, federated learning across a fleet of instrumented vehicles could enable continuous model improvement without transmitting raw audio, preserving driver privacy. The model's softmax confidence scores could support open-set anomaly detection, flagging unknown fault types as low-confidence predictions across all known classes. A BLE mesh network of Arduino-based sensors reporting to a central dashboard would enable

fleet-wide vehicle health monitoring, and integration with existing vehicle diagnostic interfaces such as OBD-II or CAN bus could provide immediate driver notification when faults are detected.

7.4 Concluding Remarks

This work demonstrates that a complete audio classification pipeline – from raw PDM microphone input through mel-spectrogram feature extraction to quantized neural network inference – can run entirely on a \$40 Cortex-M4 microcontroller with no cloud connectivity, no external accelerators, and no compromise in quantization accuracy. The M6 DS-CNN model, with only 8,166 parameters and a 21.3 KB int8 footprint, achieves an F1 macro of 0.7835 on a six-class automotive fault classification task while consuming just 33% of available flash and 68% of available SRAM. Post-training quantization is sufficient; quantization-aware training provides no additional benefit at this model scale. The primary obstacle to real-world deployment is not the classifier itself but the acoustic domain gap introduced by the evaluation methodology and the constraints of a legacy compiler toolchain – both of which are addressable through straightforward engineering effort. As edge AI hardware matures and automotive datasets grow, microcontroller-based acoustic diagnostics has the potential to become a practical, privacy-preserving, and cost-effective tool for vehicle health monitoring.

8. Author Bios

Robert Ashe (Team Leader)

Contact: rashe7414@sdsu.edu, robertashe2.0@gmail.com

LinkedIn: <https://www.linkedin.com/in/robert-ashe-nthperson/>

Robert Ashe is a master's student in the Big Data Analytics program at San Diego State University, where he also holds two paid research assistant positions at the Data-informed Construction Engineering Lab and the Fairness-Aware Multi-Agent Imitation Learning Project. His specializations span software development, machine learning and artificial intelligence, data science, and robotics. Robert is a co-founder and the sole developer of LARK Analysis Services, a company that builds AI-powered software solutions for the merchant payment card processing industry.

Rohini Panikara

Contact: rpanikara5304@sdsu.edu

LinkedIn: www.linkedin.com/in/rohinipanikara3015

Rohini Panikara is a Computer Engineering graduate student at San Diego State University with a focus on VLSI design and edge AI systems. Her current research involves developing an FPGA accelerator for energy-efficient edge AI inference. She contributed to the Arduino hardware deployment pipeline in this project.

Shivani Shankar

Contact: sshankar6566@sdsu.edu

LinkedIn: <https://www.linkedin.com/in/shivani-shankar-453076385/>

Shivani Shankar is a Computer Engineering graduate student at San Diego State University specializing in high-performance computing and applied machine learning. Her current research at the SDSU SMILE Lab focuses on building Python-based data pipelines and web portals to optimize UAV datasets for AI-driven analytics. She previously worked as a Quality Assurance Engineer at Amazon's Development Center, where she contributed to Fire OS systems and improved CI/CD pipelines to boost device reliability.

Yuvaraj Murugan

Contact: ymurugan0059@sdsu.edu

LinkedIn: <https://www.linkedin.com/in/yuvaraj-murugan-09584524b/>

Yuvaraj Murugan is a Computer Engineering graduate student at San Diego State University specializing in high-performance computing, applied machine learning, and AI. He researches autonomous UAV navigation at SDSU's SMILE Lab and has industry experience building scalable web applications and NLP-driven chatbot systems.

9. References

Abadade, Y., Temouden, A., Bamoumen, H., Benamar, N., Chtouki, Y., & Hafid, A. S. (2023). A comprehensive survey on TinyML. *IEEE Access*, 11, 96892–96922.
<https://doi.org/10.1109/ACCESS.2023.3294111>

Akbalık, F., Yıldız, A., Ertuğrul, Ö. F., & Zan, H. (2024). Engine fault detection by sound analysis and machine learning. *Applied Sciences*, 14(15), 6532.
<https://doi.org/10.3390/app14156532>

Delvecchio, S., Bonfiglio, P., & Pompoli, F. (2018). Vibro-acoustic condition monitoring of internal combustion engines: A critical review of existing techniques. *Mechanical Systems and Signal Processing*, 99, 661–683.
<https://doi.org/10.1016/j.ymssp.2017.06.033>

Global Market Insights. (2025a, December). *Predictive maintenance for vehicles market size & share 2025–2034*. <https://www.gminsights.com/industry-analysis/predictive-maintenance-for-vehicles-market>

Global Market Insights. (2025b). *Automotive predictive maintenance sensor market size, share & forecast report, 2034*. <https://www.gminsights.com/industry-analysis/automotive-predictive-maintenance-sensor-market>

Heydari, S., & Mahmoud, Q. H. (2025). Tiny machine learning and on-device inference: A survey of applications, challenges, and future directions. *Sensors*, 25(10), 3191. (p. 6) <https://doi.org/10.3390/s25103191>

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 2704–2713).
<https://doi.org/10.1109/CVPR.2018.00286>

- Krishnamoorthi, R. (2018). Quantizing deep convolutional networks for efficient inference: A whitepaper (arXiv:1806.08342) [Preprint]. arXiv.
<https://arxiv.org/abs/1806.08342>
- Lai, L., Suda, N., & Chandra, V. (2018). CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs (arXiv:1801.06601) [Preprint]. arXiv.
<https://arxiv.org/abs/1801.06601>
- Lin, J., Zhu, L., Chen, W.-M., Wang, W.-C., & Han, S. (2023). Tiny machine learning: Progress and futures. *IEEE Circuits and Systems Magazine*, 23(3), 8–34.
<https://doi.org/10.1109/MCAS.2023.3302182>
- Nasim, F., Masood, S., Jaffar, A., Ahmad, U., & Rashid, M. (2023). Intelligent sound-based early fault detection system for vehicles. *Computer Systems Science and Engineering*, 46(3), 3175–3190. <https://doi.org/10.32604/csse.2023.034550>
- Ragaie, M. (n.d.). *Car Diagnostics Dataset* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/malakraiaie/car-diagnostics-dataset>
- Suman, A., Kumar, C., & Suman, P. (2022). Early detection of mechanical malfunctions in vehicles using sound signal processing. *Applied Acoustics*, 188, 108578.
<https://doi.org/10.1016/j.apacoust.2021.108578>
- Wang, C., Song, Y., Liu, Ha., Liu, Hu., Liu, J., Li, B., & Yuan, X. (2022). Real-time vehicle sound detection system based on depthwise separable convolution neural network and spectrogram augmentation. *Remote Sensing*, 14(19), 4848.
<https://doi.org/10.3390/rs14194848>
- Zhang, Y., Suda, N., Lai, L., & Chandra, V. (2017). Hello Edge: Keyword spotting on microcontrollers (arXiv:1711.07128) [Preprint]. arXiv.
<https://arxiv.org/abs/1711.07128>

10. Appendix

10.1 GitHub Repository

Car Sounds Classification: https://github.com/nthPerson/Car_Sounds_Classification

10.2 Repository Structure

The annotated directory tree below highlights the parts of the repository most relevant for review. Items marked [*] are recommended starting points and key components.

```
.
|-- GETTING_STARTED.md          # Team onboarding: env setup + end-to-end project run order
|-- README.md                  # Project overview, motivation, quickstart
|-- requirements.txt            # Python dependencies (TensorFlow, librosa, sklearn)
```

```

|-- notebooks/                                # [*] START HERE -- run in numeric order
| |-- 01_preprocessing.ipynb                  # Audio loading, resampling, mel/MFCC feature gen
| |-- 02_classical_ml.ipynb                  # M1 baselines: Random Forest + SVM (441-dim feats)
| |-- 03_neural_networks.ipynb               # M2/M5/M6 float32 training across all 3 tiers
| |-- 04_noise_analysis.ipynb                # Noise-bank characterization, SNR study
| |-- 05_noise_augmentation.ipynb            # Noise-augmented training data generation
| |-- 06_quantization.ipynb                  # PTQ + QAT (M3/M4) and quantization-impact analysis
| \-- 07_evaluation.ipynb                    # Final evaluation, paper figures, playback test

|-- docs/
| |-- project_spec/                          # Design specs (the "what & why")
| | |-- project_overview.md                  # Goals, scope, hardware constraints, taxonomy tiers
| | |-- data_preprocessing_spec.md           # Feature pipeline, augmentation, splits
| | |-- model_architecture_spec.md           # M1--M6 architectures, hyperparameters
| | |-- quantization_deployment_spec.md      # PTQ/QAT methods, Arduino sketch design
| | |-- evaluation_plan.md                   # Metrics, quantization impact, on-device benchmarks
| | |-- development_roadmap.md               # 8-phase plan with dependencies and risk
| | |-- deployment_feasibility_report.md     # Pre-project hardware feasibility analysis
| | \-- dev_log.md                           # Chronological development log (newest first)

|-- src/                                     # Python modules used by notebooks and scripts

|-- arduino/
| |-- car_sound_classifier/                  # Code that runs on the Arduino Nano 33 BLE Sense Rev2
| \-- noise_collector/                      # [*] Final deployment sketch
|                                           # Sketch used to record the realistic ambient-noise bank

|-- car_diagnostics_dataset/                 # Raw audio: 1,386 WAV files, 13 conditions

|-- data/                                   # Processed data + frozen splits

|-- models/                                 # Trained model artifacts

\-- results/                               # [*] Evaluation artifacts referenced in the report

```